Semantic Video Package (SVP) v1.0 Release Candidate 2

A strict open standard for storing machine-readable observations of time-based media

Status: Release Candidate 2, OCR and structured color observation revision
Date: 2026-06-19
Editor: SVP Working Draft
Specification license: CC0-1.0
Reference implementation license: Apache-2.0
Package extension: .svp
Detached signature extension: .svpsig
Reference model bundle extension: .svpmodel
MIME type target: application/vnd.svp+zip

Revision history

Draft	Date	Notes
0.1	2026-06-18	First standards-style draft defining SVP Core, package layout, reference stack, validation rules, query model, governance, and conformance classes.
0.2	2026-06-18	Adds reference execution DAG, BLAKE3 content-addressable memoization, resumable recovery journal, SVP binary block headers, package compression rules, and stricter validation for binary payloads.
0.3	2026-06-18	Clarifies deterministic equivalence, adds optional overlapping-speech word metadata, defines valid degenerate visual-source outputs, adds reader/validator release sequencing guidance, and standardizes forward compatibility for future label schemas.
0.4	2026-06-18	Resolves canonical raster policy for non-16:9 media, defines logical SQLite

Draft	Date	Notes
		equivalence, adds cache and journal lifecycle rules, reserves binary block type codes, replaces transcript language strings with a stable language object, and standardizes machine-readable validation reports and codes.
0.5	2026-06-19	Resolves the SQLite equivalence contradiction for tolerance-governed payloads and vectors, requires even canonical raster dimensions, and expands cross-platform fixture testing to verify payload and SQLite equivalence together.
0.6	2026-06-19	Defines exact rational timestamp rounding, adds optional detached `.svpsig` authenticity sidecars, requires a machine-readable validation-code registry, requires per-layer equivalence metrics, keeps word-level language out of v1.0 Core, and defines native SVP Model Bundles for reference model distribution without Python runtime requirements.
RC1	2026-06-19	Resolves final release-candidate blockers: defines `index/index_manifest.json`, separates authenticity findings from Core status and exit codes, unifies canonical model IDs, expands the model-bundle manifest contract, and adds `index-manifest.schema.json`.
RC2	2026-06-19	Adds first-class OCR/visible-text observations and structured color observations to SVP Core; defines package paths, schemas, registries, validation codes, index/query tables, provenance rules, builder roadmap changes, fixture requirements, and implementation pause guidance before Phase 03+.

Normative artifacts for v1.0

A complete SVP v1.0 release should ship these artifacts together in one public repository:

```
/spec/svp-v1.md
/spec/validation-codes.json
/spec/equivalence-profile.json
/spec/block-types.json
/spec/color-buckets.json
/spec/color-spaces.json
/spec/ocr-observation-types.json
/spec/svp-model-bundle-v1.md
/spec/svp-signature-v1.md
/schemas/*.schema.json
/reference/
/conformance/
/fixtures/
/models/
/samples/
/docs/
```

This document is the release-candidate seed for `/spec/svp-v1.md`.

`/spec/validation-codes.json`, `/spec/equivalence-profile.json`, `/spec/block-types.json`, `/spec/color-buckets.json`, `/spec/color-spaces.json`, `/spec/ocr-observation-types.json`, `/spec/svp-model-bundle-v1.md`, and `/spec/svp-signature-v1.md` are normative machine-readable companions to this document. `index-manifest.schema.json`, `text-region.schema.json`, `text-observation.schema.json`, `numeric-value.schema.json`, `text-absence.schema.json`, `color-observation.schema.json`, `color-summary.schema.json`, and `color-absence.schema.json` are required in `/schemas`. If a registry file, schema file, and the prose conflict, the release is defective and MUST be corrected before a final v1.0 tag.

RC2 change summary

RC2 is a release-candidate revision that adds two observation layers to SVP Core: OCR/visible text and structured color observations. It does not weaken any existing required section, required field, validator behavior, conformance class, timestamp rule, binary block rule, equivalence rule, or provenance rule.

RC2 makes six focused changes:

1. It makes visible text, OCR text regions, recognized text, and safely parseable numeric values first-class observations rather than labels.
2. It makes measurable color distributions first-class observations for scenes, shots, frames, regions, entities, and text regions.
3. It defines required package paths under `/text/` and `/colors/`, plus required schemas and registries for OCR observation types, color spaces, and color buckets.
4. It expands the SQLite index and query model so readers can search visible text, normalized visible

text, numeric values, color bucket coverage, text-region foreground/background colors, and color coverage over time.

5. It expands validation behavior and validation-code registries so OCR and color records are schema-valid, reference-valid, provenance-backed, indexed, and queryable.
6. It updates the builder and implementation roadmap so Phase 03+ work pauses until OCR and color are wired into the validator, fixtures, index model, builder stages, and first-video trial acceptance checks.

The core principle is unchanged:

```
SVP Core is strict.
OCR text is observation.
Measured color distribution is observation.
Labels are interpretation.
A visible string is not a semantic label.
A color percentage is not a claim about real-world material color.
A validator must be able to implement the spec without guessing.
```

1. Executive summary

SVP is a media package standard that turns video into a structured, queryable, agent-readable artifact. A normal video container stores pixels and sound over time. SVP stores the original video and audio plus the required observation layers a machine needs in order to reason about the media without repeatedly decoding and reanalyzing every frame.

The core idea is intentionally blunt:

```
MP4 = pixels and audio over time
SRT = words over time
SVP = words, speakers, shots, scenes, entities, masks,
    depth, OCR text, color observations, embeddings,
    relationships, and indexes over time
```

SVP is not an AI product. SVP is not a video editor. SVP is not a folder of helper files. SVP is a standard package format.

The reference processor can use classical signal processing, classical computer vision, and bundled local models to produce required observations. The standard itself gives no special status to any AI vendor, cloud API, model provider, or editing app.

The v1.0 principle is strict:

If a package does not contain the required core layers, it is not an SVP package.

SVP Core requires all of the following:

```
Original Video
Original Audio
Transcript
Word Timestamps
Speaker Segments
Shot Boundaries
Scene Boundaries
Entity Tracks
Spatial Regions
Relationships
Search Index
Depth
Masks
Embeddings
OCR / Visible Text
Numeric Values
Structured Color Observations
Manifest
```

Only one section is outside core:

```
Labels
```

Labels are non-core because labels are interpretation. Everything else is the observation substrate.

2. Normative language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** are to be interpreted as described in RFC 2119 and RFC 8174 when written in uppercase.

Lowercase words such as "may" and "should" are ordinary English.

3. Design philosophy

3.1 The standard is not the processor

The SVP standard defines what must exist in a valid package, how it is encoded, how it maps to time, how it is indexed, and how readers validate it.

A processor is merely one implementation that produces a valid package. A package can be produced by a CLI, desktop app, render farm, camera, archive system, local model, or human-assisted tool. It is valid only if the required SVP data exists and validates.

3.2 Observation is separate from interpretation

SVP Core stores observations:

```
Entity track exists.
Entity appears at these coordinates.
Entity has this mask.
Entity has this relative depth distribution.
Entity overlaps another entity.
Speaker 0001 is active during this interval.
Word 538 starts at 84.120 seconds.
Text region 92 reads "$12.99" during shot 14.
Scene 004 has orange bucket coverage of 0.70 under the registered color-bucket rules.
```

SVP Labels store interpretations:

```
Entity 84 is an iPhone.
Speaker 0001 is Dom.
This shot is a product beauty shot.
The visible string "$12.99" means this is an advertisement.
This scene feels warm.
```

The first group is core. The second group is labels.

3.3 Everything attaches to time

Every meaningful record in SVP maps to the presentation timeline. Words, speaker segments, shots, scenes, entity regions, masks, depth frames, embeddings, and relationships all exist over time.

SVP does not use vague timestamps. The normative time representation is integer microseconds from the start of the primary presentation timeline.

3.4 Everything has provenance

Every generated layer MUST include provenance. A reader must be able to answer these questions:

```
What generated this record?
Which version generated it?
Which model or algorithm was used?
Which input records were used?
What confidence is attached?
Can this layer be rebuilt?
```

3.5 No remote dependency is required for conformance

A conforming SVP builder MUST NOT require a cloud AI API to generate a conforming package. The

official reference builder is local-first.

3.6 The package is portable

An SVP file MUST be a single file for interchange. Internally, it is a ZIP64 package with a fixed layout, mandatory UTF-8 JSON and JSONL metadata, binary spatial chunks, and a mandatory SQLite index.

3.7 Labels are the only non-core section

SVP v1.0 has no core extension system and no plugin mechanism. A valid file contains the required core. Unknown root-level sections cause strict validation failure, except `/labels`.

RC2 clarifies that `/text/` and `/colors/` are not labels. A visible string in a frame and a measured color distribution over pixels are observations. The package MUST NOT require a semantic interpretation such as "advertisement", "sunset", "warning", "brand", or "danger" for Core validity.

4. Scope and non-goals

SVP v1.0 defines:

1. The package container.
2. Required media assets.
3. Required transcript and word timing schema.
4. Required speaker segment schema.
5. Required shot and scene schema.
6. Required entity track schema.
7. Required spatial region schema.
8. Required depth and mask encodings.
9. Required relationship graph schema.
10. Required embedding schema.
11. Required OCR and visible-text schema.
12. Required structured color-observation schema.
13. Required search index schema.
14. Required provenance and validation rules.
15. A fixed reference processing stack.
16. A read-only query interface for apps and agents.
17. Open source governance and conformance expectations.

SVP v1.0 does not define:

1. A video editor UI.
2. A cloud service.
3. A vendor marketplace.
4. A model marketplace.

5. A general plugin system.
6. A required labeling model.
7. A replacement for Premiere, Final Cut, or Resolve project files.
8. A replacement for OpenTimelineIO.

OpenTimelineIO remains the chosen export target for editorial timeline interchange. SVP stores observations. OTIO stores editorial cut information.

5. Chosen reference stack

This section is concrete on purpose. These are the stack choices for the official reference implementation. There are no parallel alternatives in the reference stack.

Third-party builders can use different internals, but a third-party package is conforming only if the final `.svp` file passes strict validation.

5.1 Implementation language

Choice: C++20

Build system: CMake 3.28+
Dependency manager: vcpkg manifest mode
CLI parser: CLI11
Test framework: Catch2
Desktop app later: Qt 6 using the same C++ core library

Reasoning:

1. FFmpeg, OpenCV, ONNX Runtime, SQLite, zstd, and archive libraries all expose stable C or C++ APIs.
2. C++ avoids a fragile FFI core for high-volume video processing.
3. The same core can ship as a CLI, library, desktop app backend, and server process.
4. CMake plus vcpkg gives cross-platform builds for Windows, macOS, and Linux.

Reference repository layout:

```
svp/
  spec/
    svp-v1.md
    schemas/
  reference/
    src/
      svp_core/
      svp_build/
      svp_validate/
      svp_query/
```

```
    svp_export/
  cli/
    svp.cpp
  tests/
    conformance/
    golden/
conformance/
samples/
docs/
```

5.2 Media ingest

Choice: FFmpeg 8.1.x libraries and ffprobe-compatible stream inventory.

SVP uses FFmpeg for demuxing, decoding, stream inventory, timecode extraction, audio extraction, frame access, pixel format normalization, and derived analysis assets.

The reference processor MUST record exact FFmpeg library versions in `provenance/build.json`.

FFmpeg is chosen because it is the common open media plumbing across platforms. ffprobe is used as the behavioral baseline for machine-readable stream inventory.

5.3 Classical computer vision

Choice: OpenCV 4.x C++ API

OpenCV is used for:

```
Color conversion
Feature detection
Sparse optical flow
Dense optical flow
Global camera motion estimation
RANSAC homography estimation
Kalman filtering
Background subtraction
Region generation
GrabCut-style mask refinement
Contour extraction
Morphological cleanup
```

The reference visual tracker is not a labeler. It produces persistent visual entities using motion, appearance, depth, and mask coherence.

5.4 Shot and scene boundaries

Choice: Native C++ implementation of PySceneDetect-style detector families.

SVP does not embed PySceneDetect as a runtime dependency. The reference processor implements the same detector families in C++:

```
HSV content delta for hard cuts
Adaptive rolling-delta thresholding for fast motion
RGB intensity thresholding for fades and black frames
Y-channel histogram deltas for abrupt changes
Minimum scene length enforcement
```

Shot boundaries are low-level visual edits. Scene boundaries group shots into larger units using shot adjacency, visual similarity, audio continuity, speaker continuity, and transcript embedding changes.

5.5 Speech detection

Choice: Silero VAD ONNX through ONNX Runtime.

Silero VAD finds speech regions before transcription and diarization. The reference processor uses the ONNX path, not a PyTorch runtime. Speech intervals are written to `transcript/speech_regions.jsonl`.

5.6 Transcription and word timestamps

Choice: whisper.cpp with bundled local model `ggml-large-v3-turbo-q5_0.bin`.

The reference processor uses whisper.cpp for local transcription and word-level timestamps. The package stores word timings in integer microseconds and human-readable seconds strings. Integer microseconds are normative.

5.7 Speaker diarization

Choice: sherpa-onnx speaker diarization models.

The reference processor uses sherpa-onnx for speaker segmentation and speaker embeddings. Speaker IDs in SVP Core are anonymous:

```
speaker_0001
speaker_0002
```

Human names are labels, not core.

5.8 Depth

Choice: Depth Anything V2 Small, Apache-2.0 model only.

Depth is required in SVP Core. The reference processor stores relative inverse depth maps at the canonical SVP analysis raster.

Depth Anything V2 Base, Large, and Giant are not used in the official reference processor because their

published model license is noncommercial. The Small model is selected because it is published as Apache-2.0.

Depth in SVP v1.0 is relative, not metric. A value tells the machine what is closer or farther inside the observed frame. It does not claim real-world meters unless a future major version standardizes metric camera calibration.

5.9 Masks

Choice: OpenCV classical mask pipeline.

Masks are required. The reference processor produces masks without object labels using:

```
Shot-local keyframe selection
Sparse KLT point tracks
Dense Farneback optical flow
Global camera motion compensation
Residual motion clustering
Depth-edge assisted segmentation
Color and texture region grouping
GrabCut refinement seeded by region boxes
Mask propagation through optical flow
Kalman-smoothed box and centroid tracking
Run-length encoded mask storage
```

The mask pipeline does not need to know what an entity is. It only needs to track coherent visible regions through time.

5.10 Embeddings

Choice: Nomic Embed Text v1.5 and Nomic Embed Vision v1.5.

SVP v1.0 requires embeddings. The reference processor uses Nomic text and vision embeddings because they are open, local, Apache-2.0, 768-dimensional, and designed to share a text and vision latent space.

Embeddings are stored as normalized float32 vectors. All embedding records MUST include `model_id`, `model_bundle_id`, `model_blake3`, `dim`, `normalization`, and `input_ref`.

The reference model set MUST pin the exact model revision, exact files, BLAKE3 hashes, license text, preprocessing contract, postprocessing contract, and output dimensionality. The builder MUST NOT fetch a moving `latest` model during a build.

5.11 Inference runtime

Choice: ONNX Runtime C++ API for ONNX/ORT model artifacts, plus native reference runtimes where already chosen by the spec.

ONNX Runtime is used for local VAD, diarization, text embeddings, vision embeddings, and depth when

model artifacts are distributed as ONNX or ORT in the reference build. The reference implementation MUST expose the execution provider used for each emitted artifact in provenance.

Execution provider order in the reference processor:

```
macOS: CoreML if available, then CPU
Windows: DirectML or WinML path if available, then CPU
Linux: CUDA if explicitly installed, then CPU
All platforms: CPU fallback is mandatory
```

Conformance does not require accelerated hardware.

5.11.1 Reference Model Bundles

RC1 defines the **SVP Model Bundle** as the required distribution unit for models used by the reference builder. The goal is simple: developers should not need to create local Python environments, run arbitrary model-provider code, chase transitive package versions, or manually place weights in hidden folders merely to build an SVP package.

A reference model bundle uses extension `.svpmodel` and is a ZIP64 package with this required layout:

```
model.svpmodel.json
files/
  model.onnx
  model.ort
  tokenizer.json
  tokenizer_config.json
  preprocessor.json
  postprocessor.json
LICENSE
NOTICE
```

Only files actually required by a bundle need to exist, but `model.svpmodel.json`, `LICENSE`, and `NOTICE` MUST exist. Exactly one `model-lock.json` MUST exist at the model-cache root and MUST NOT be duplicated inside a bundle.

5.11.1.1 Canonical model identity

All normative SVP model references use canonical SVP-native model IDs with the `model_` prefix defined in Section 8.

Examples:

```
model_depth_anything_v2_small
model_whisper_large_v3_turbo_q5_0
model_nomic_embed_text_v1_5
model_nomic_embed_vision_v1_5
```

The following fields MUST use canonical SVP model IDs:

```
model.svpmodel.json.model_id
model-lock.json model references
embedding_sets.json.model_id
provenance/model_hashes.jsonl.id
provenance/processors.jsonl.model_refs[]
DAG task model_refs[]
cache-key model_id inputs
validation and equivalence model identity fields
```

Registry slugs such as `nomic-ai/nomic-embed-text-v1.5`, repository names, download URLs, and provider-specific identifiers MAY be recorded only as source metadata. They MUST NOT be used as normative model IDs, cache-key model IDs, provenance references, or embedding-set IDs.

5.11.1.2 Model bundle ID

A model bundle ID identifies one exact packaged model distribution, not just a model family. It is derived from the canonical model ID, model version, and the BLAKE3 of the canonical model bundle manifest plus referenced file hashes.

The required form is:

```
<model_id>@<model_version>+blake3_<first_12_hex_of_bundle_blake3>
```

Example:

```
model_depth_anything_v2_small@2024-06-13+blake3_8f4c21aa93d0
```

The `bundle_blake3` field stores the full 64-character BLAKE3 digest. The shortened suffix in `model_bundle_id` is an identifier convenience and MUST NOT replace full hash verification.

5.11.1.3 Required model manifest example

A model bundle manifest MUST include every field shown below unless a field is explicitly marked optional by the `model-bundle.schema.json` schema.

```
{
  "schema_version": "svp-model-bundle-v1",
  "model_bundle_id": "model_depth_anything_v2_small@2024-06-13+blake3_8f4c21aa93d0",
  "model_id": "model_depth_anything_v2_small",
  "model_version": "2024-06-13",
  "bundle_blake3": "blake3:8f4c21aa93d0...",
  "display_name": "Depth Anything V2 Small",
  "source_registry": "github",
  "source_slug": "DepthAnything/Depth-Anything-V2",
```

```
"source_revision": "2024-06-13-pinned",
"runtime": "onnxruntime",
"format": "onnx",
"license": "Apache-2.0",
"supported_execution_providers": [
  "cpu",
  "coreml",
  "cuda",
  "directml"
],
"files": [
  {
    "path": "files/model.onnx",
    "role": "weights",
    "blake3": "blake3:..."
  },
  {
    "path": "files/preprocessor.json",
    "role": "preprocessor_contract",
    "blake3": "blake3:..."
  },
  {
    "path": "files/postprocessor.json",
    "role": "postprocessor_contract",
    "blake3": "blake3:..."
  },
  {
    "path": "LICENSE",
    "role": "license",
    "blake3": "blake3:..."
  },
  {
    "path": "NOTICE",
    "role": "notice",
    "blake3": "blake3:..."
  }
],
"input_contract": {
  "canonical_raster_required": true,
  "input_space": "canonical_analysis_raster",
  "color_space": "rgb",
  "dtype": "float32",
  "layout": "NCHW"
},
"output_contract": {
  "type": "depth",
  "dtype": "float32",
  "shape": ["1", "1", "H", "W"],
```

```
    "canonical_output_extent": "canonical_analysis_raster"
  },
  "preprocessor_contract": {
    "resize": "already_canonical_analysis_raster",
    "color_order": "rgb",
    "layout": "NCHW",
    "normalization": {
      "mean": [0.485, 0.456, 0.406],
      "std": [0.229, 0.224, 0.225]
    }
  },
  "postprocessor_contract": {
    "output_type": "relative_depth",
    "normalization": "svp_depth_v1_per_frame_normalized",
    "dtype": "float32",
    "extent": "canonical_analysis_raster"
  }
}
```

5.11.1.4 Model lock authority

`model.svpmodel.json` is authoritative for the contents, identity, contracts, license, notice, and file hashes of a single model bundle.

The single model-cache-root `model-lock.json` is authoritative for the complete reference model set used by a builder release or build invocation. It records which exact model bundles are selected together and repeats every required file path, role, and BLAKE3 value from each selected manifest.

If `model-lock.json` and any referenced `model.svpmodel.json` disagree on `model_id`, `model_version`, `model_bundle_id`, `bundle_blake3`, file paths, file roles, or file BLAKE3 values, the builder MUST reject the model set before processing media.

Reference Model Bundle rules:

1. The reference builder MUST support installing, verifying, and using pinned `.svpmodel` bundles.
2. The reference builder MUST NOT require Python for normal end-user operation.
3. The reference builder MUST NOT execute remote model code during normal operation.
4. Every model file MUST be content-addressed with BLAKE3 in `model.svpmodel.json` and `model-lock.json`.
5. Every model bundle MUST include complete license and notice files.
6. Every model bundle MUST declare runtime, format, input contract, output contract, preprocessor contract, postprocessor contract, and supported execution providers.
7. The builder MUST verify bundle hashes before use.
8. The builder MUST record `model_bundle_id`, `bundle_blake3`, individual file BLAKE3 values, runtime, execution provider, and canonical `model_id` in provenance.
9. The builder MUST NOT auto-update a bundle during a build.
10. Offline installers MAY include the complete required reference model set.

11. Installation MUST assemble the complete set in a separate staging directory, verify the root lock against every selected bundle, and only then publish the staged cache transactionally.

Required model-management commands are defined in Section 21.

5.12 Package container and compression

Choice: ZIP64 package with SVP binary block streams.

SVP interchange files are ZIP64 archives with extension `.svp`. The first entry MUST be an uncompressed `mimetype` file containing exactly:

```
application/vnd.svp+zip
```

The package container is responsible for containment. It is not responsible for compressing already-compressed SVP binary payloads.

Required package compression rules:

1. `mimetype` MUST use ZIP method `STORE`.
2. `spatial/*.svpdz`, `spatial/*.svpmz`, and `embeddings/*.svpez` MUST use ZIP method `STORE`.
3. The SVP binary block payload inside those files MUST use the compression declared in the SVP block header. RC1 defines Zstandard as compression code `0x01`.
4. JSON, JSONL, and manifest entries MAY use ordinary ZIP compression.
5. `index/index.sqlite` SHOULD use ZIP method `STORE` so readers can extract or memory-map it without redundant decompression.
6. `media/original/*` SHOULD use ZIP method `STORE`. Already-compressed media formats (MP4, MOV, FLAC, etc.) gain negligible size from Deflate, and STORED entries enable `zip_fseek` random access for streaming readers that read media directly from the ZIP without extracting to disk.
7. A file whose bytes begin with the SVP binary block magic `SVPB` MUST NOT use a `.zst` extension.

Large depth, mask, and embedding payloads are therefore not plain Zstandard files. They are SVP block streams whose payload sections are Zstandard-compressed.

5.13 Search index

Choice: SQLite with FTS5 and sqlite-vec.

SVP v1.0 requires `index/index.sqlite`. It is not a cache. It is part of the package because readers and agents need fast local query without rebuilding indexes.

The index contains:

```
Full-text search over transcript, shots, scenes, relationships,
and provenance text
Vector search over transcript chunks, shots, scenes, visual regions,
```

```
and entity crops
Temporal lookup tables
ID lookup tables
Relationship lookup tables
Binary block lookup tables
Integrity hashes
```

SQLite FTS5 is the required text index. sqlite-vec is the required vector table mechanism in the reference implementation. sqlite-vec is vendored and pinned so upstream pre-v1 changes do not leak into SVP compatibility.

5.14 Timeline export

Choice: OpenTimelineIO for editorial timeline exports.

SVP is not an editorial timeline format. The reference tool exports OTIO when an app or agent needs to create an edit timeline.

```
svp export video.svp --format otio --out roughcut.otio
```

5.15 Hashing and content-addressed identity

Choice: BLAKE3, 256-bit digest, lowercase hexadecimal encoding.

All SVP-native integrity hashes, cache keys, block hashes, package hashes, artifact hashes, model-bundle hashes, detached-signature package hashes, and processor output hashes MUST use BLAKE3. The canonical field name is `blake3`. RC1 continues to avoid mixing `sha256` into core records.

External ecosystems sometimes publish SHA-256 checksums for model weights or source downloads. SVP MAY record those values under `alternate_hashes`, but they are not normative for SVP integrity.

Required hash rules:

1. BLAKE3 output is exactly 32 bytes.
2. JSON and JSONL records store BLAKE3 as 64 lowercase hexadecimal characters.
3. Binary headers store BLAKE3 as raw 32-byte values.
4. Hash inputs MUST be byte-exact and documented in provenance.
5. Builders MUST NOT reuse cached artifacts unless the cache key and artifact hash both verify.

5.16 Reference builder orchestration and package equivalence

Choice: deterministic asynchronous DAG execution with defined package equivalence.

The reference builder is a media compiler. It MUST execute the pipeline as an asynchronous directed acyclic graph so independent audio, vision, indexing, and packaging stages can run concurrently.

The standard does not mandate a specific scheduler library. The reference implementation MUST expose

deterministic task boundaries, task dependencies, resumable task IDs, cache keys, and provenance for every emitted artifact.

A conforming package MUST be independent of task completion order. JSONL row order, ID assignment, index records, provenance records, and binary block references MUST be written in canonical order by section-specific sort keys, not by worker completion order.

5.16.1 Equivalent package contents

RC1 defines **equivalent package contents** because bit-exact equality is not realistic for every derived observation across threads, CPUs, GPUs, accelerators, math libraries, and ONNX Execution Providers.

Two packages are eligible for equivalence comparison only when they are produced from the same canonical source media, the same SVP schema version, the same processor contracts, the same model artifacts, the same processor parameters, and the same normalized inputs. A package produced by a different processor contract, a different model artifact, a different transcript output, or different source media is not equivalent merely because it looks similar.

Equivalent package contents are evaluated in three categories:

```
Strict byte equivalence:
  original media bytes
  lossless extracted audio bytes
  canonical JSON and JSONL structure
  IDs and references
  shot and scene frame indices
  source fingerprints
  package layout

Logical data equivalence:
  SQLite index contents after SVP canonical logical row-stream serialization

Numerical observation equivalence:
  depth arrays
  embedding vectors
  mask boundaries
  Kalman-smoothed track coordinates
  optical-flow-derived coordinates
  confidence values emitted by floating-point processors
```

For strict byte equivalence, the canonical BLAKE3 digests MUST match exactly.

For logical data equivalence, physical storage details that do not affect query results are ignored. SQLite page size, free lists, insertion history, vacuum state, journal history, rowid allocation side effects, and byte order of unrelated internal pages are not package-equivalence facts. The normative comparison input is the SVP canonical logical comparison procedure defined in Section 17.5.

RC1 adds one additional rule: the SQLite equivalence path MUST preserve the same equivalence class as

the source value represented by each SQLite column. Exact-match source values remain exact-match inside SQLite. Tolerance-governed source values remain tolerance-governed inside SQLite. A validator MUST NOT accidentally convert a numerically equivalent depth, mask, embedding, or floating-point observation into `not_equivalent` merely because an index table stores a hash or vector derived from that observation.

For numerical observation equivalence, the package structure, IDs, references, time ranges, frame ranges, dimensions, dtype declarations, provenance references, and record counts MUST match exactly. The numeric payloads then MUST satisfy the Default Equivalence Profile defined below.

5.16.2 Default Equivalence Profile v1

The Default Equivalence Profile v1 is used by the reference validator when comparing two SVP packages for equivalence. It is not used to validate whether a single package is complete. A single package remains valid only when every required core section exists and passes strict structural validation.

Layer	Equivalence rule
Original media	Exact BLAKE3 match.
Lossless extracted audio	Exact BLAKE3 match.
Canonical JSON and JSONL	Exact canonicalized BLAKE3 match after normalizing UTF-8, object key order, and line endings according to the schema canonicalization rules.
SQLite index	Source-layer-aware logical reproducibility using the canonical logical comparison procedure defined in Section 17.5. Raw `index.sqlite` bytes are not compared for package equivalence. Exact columns remain exact. Tolerance-governed payloads and vectors use their source-layer equivalence rules.
IDs and references	Exact string match.
Shot boundaries	Exact frame-index match. Human-readable seconds strings are ignored when integer microseconds and frame indices match.
Scene boundaries	Exact referenced shot IDs and

Layer	Equivalence rule
	integer time ranges.
Depth maps	Same block structure, dimensions, dtype, frame range, and time range. Decoded normalized depth values MUST have mean absolute error <= 0.002, p99 absolute error <= 0.010, and Spearman rank correlation >= 0.995 per block.
Embeddings	Same embedding set, model identity, dimension, input reference, and vector count. Corresponding normalized vectors MUST have cosine similarity >= 0.999.
Masks	Same mask record structure, dimensions, frame range, and target references. Non-empty corresponding masks MUST have IoU >= 0.995 and p95 boundary displacement <= 1.0 canonical analysis raster pixel. Empty masks are equivalent only to empty masks.
Entity tracks	Same track IDs, keyframe frame indices, and region references. Corresponding bounding boxes MUST have IoU >= 0.995; centroid displacement MUST be <= 1.0 canonical analysis raster pixel.
Kalman-smoothed coordinates	Same keyframe structure. Coordinate absolute error MUST be <= 1.0 canonical analysis raster pixel.
Confidence fields	Absolute difference MUST be <= 0.001 unless the processor record declares a tighter tolerance.
OCR text regions	Same IDs, timing, target references, and schema structure. Bounding

Layer	Equivalence rule
	boxes MUST have IoU >= 0.995; confidence fields use the confidence tolerance.
Text observations	raw_text and normalized_text MUST match exactly when produced from the same reference model set and OCR normalization rules. Confidence fields use the confidence tolerance.
Numeric values	Decimal-string numeric values MUST match exactly. Raw source text MUST match exactly.
Color observations	Same target type, target ID, sampling basis, color space, and bucket registry version. Corresponding bucket coverage values MUST differ by <= 0.005; coverage totals MUST sum within 0.001; dominant bucket MUST match unless buckets are tied within tolerance.

The reference validator MUST report whether two packages are:

```
byte_identical
structurally_equivalent
numerically_equivalent
not_equivalent
```

Reporting rules:

1. byte_identical means the package bytes match exactly, including the physical index.sqlite file.
2. structurally_equivalent means all exact-governed package structure, IDs, references, time ranges, schema fingerprints, FTS text, relationship rows, and non-tolerance-governed logical SQLite rows match, but package bytes are not identical.
3. numerically_equivalent means all exact-governed structure matches and one or more tolerance-governed payloads, vector values, or floating-point observations differ only within the Default Equivalence Profile thresholds.
4. not_equivalent means an exact-governed field differs, a referenced source layer fails its numerical equivalence rule, a required mapping is missing, or the packages were not eligible for equivalence comparison.

BLAKE3 hashes are never tolerance-compared. A hash either matches or does not. However, a mismatch in a SQLite column that stores a BLAKE3 hash of a tolerance-governed source payload MUST NOT by itself force `not_equivalent` when the referenced source payloads satisfy the applicable numerical equivalence rule.

A package can be numerically equivalent without being byte-identical. That is not a validation failure.

5.16.3 Cache determinism is stricter than package equivalence

Package equivalence MUST NOT be used to justify unsafe cache reuse. Cache reuse remains exact. A cached artifact may be reused only when its cache key and artifact BLAKE3 verify exactly. Numerical equivalence is for comparing finished packages, not for accepting stale cache artifacts.

5.17 OCR and visible text

SVP RC2 makes OCR and visible text a required observation layer for video packages. The reference builder MUST include a text detection, text recognition, layout classification, and numeric extraction stage.

The reference stack SHOULD implement OCR through native model bundles, not ad-hoc Python runtime environments. The pipeline is divided into these processor boundaries:

```
text_detection
text_recognition
text_layout_classification
numeric_extraction
text_region_color_sampling
text_indexing
```

A conforming package does not claim that visible text is semantically meaningful. It only records that text-like pixels were detected, recognized, normalized, timed, located, and indexed.

5.18 Structured color observations

SVP RC2 makes structured color observations required for video packages. Color processing is a deterministic observation layer and does not require an AI model.

The reference builder MUST compute color distributions from the canonical analysis raster and from registered sampling targets. It MUST use a registered color space and a registered color-bucket version. It MUST record the sampling basis, target record, bucket coverage ratios, dominant bucket, quality score, and provenance.

The reference stack MUST treat color as measured pixel distribution only. It MUST NOT claim real-world material color. Lighting, compression, HDR transforms, white balance, shadows, transparency, and reflections can all shift measured pixel color.

6. Package layout

A valid SVP v1.0 package MUST contain exactly the following core sections and files. Unknown root-level sections MUST cause strict validation failure, except `/labels`, which is the only non-core section allowed by v1.0.

```
video.svp
 mimetype
 manifest.json

 media/
  original/
   source_000.ext
  audio/
   original_stream_000.flac
   analysis_mono_16k.wav
   waveform.jsonl
   audio_absence.json

 transcript/
  speech_regions.jsonl
  transcript.json
  words.jsonl
  speakers.jsonl
  speaker_segments.jsonl

 timeline/
  frames.jsonl
  shots.jsonl
  scenes.jsonl

 entities/
  entities.jsonl
  entity_tracks.jsonl

 spatial/
  regions.jsonl
  masks.index.jsonl
  masks.blocks.svpmz
  depth.index.jsonl
  depth.blocks.svpdz

 text/
  text_regions.jsonl
  text_observations.jsonl
  numeric_values.jsonl
  text_absence.json
```

```
colors/
  color_observations.jsonl
  color_summary.json
  color_absence.json

relationships/
  relationships.jsonl

embeddings/
  embedding_sets.json
  embeddings.index.jsonl
  embeddings.blocks.svpez

index/
  index.sqlite
  index_manifest.json

provenance/
  build.json
  processors.jsonl
  input_hashes.jsonl
  model_hashes.jsonl
  validation.json

labels/
  labels.jsonl
```

`labels/labels.jsonl` is the only non-core file tree. It MAY be absent. If present with a supported label schema version, it MUST validate against that label schema. If present with an unsupported future label schema version, it MUST be treated according to Section 19.1 and MUST NOT invalidate SVP Core.

`text_absence.json` and `color_absence.json` MUST exist. For normal video media, they record that OCR and color processors ran even if no text regions were found or if color observations are near-uniform. `audio_absence.json` MUST exist. For normal media it records `source_audio_present: true`. For silent source media it records `source_audio_present: false`, and `original_stream_000.flac` MUST be a canonical silent FLAC asset with provenance. This keeps the audio section required without making silent video impossible.

The recovery journal, global cache, temporary task outputs, and scheduler state are not package sections. They MUST NOT appear inside the final `.svp` archive.

7. Canonical time model

All core timestamps use integer microseconds from the primary presentation start.

Required timing fields for span records:

```
{
  "start_us": 84 120 000,
  "end_us": 85 260 000,
  "start_sec": "84.120",
  "end_sec": "85.260"
}
```

Rules:

1. start_us is inclusive.
2. end_us is exclusive.
3. end_us MUST be greater than start_us for non-instant records.
4. start_sec and end_sec are decimal strings derived from microseconds.
5. The normative value is the integer microsecond value.
6. Floating-point seconds MUST NOT be used as normative timestamps.
7. Frame records MUST include frame_index, pts_us, and source_stream_id.
8. If the source contains non-zero start time, SVP normalizes the primary presentation start to 0 and records the original offset in provenance.
9. Builders MUST derive canonical timestamps from rational source timebases using exact integer rational arithmetic.
10. Builders MUST NOT use floating-point arithmetic to compute canonical pts_us values.
11. The final rational-to-integer microsecond conversion MUST use deterministic round-half-to-even.

For a source presentation timestamp pts and a source timebase timebase_num / timebase_den, the conceptual calculation is:

```
exact_value = pts * timebase_num * 1_000_000 / timebase_den
pts_us = round_half_to_even(exact_value)
```

round_half_to_even means:

1. If the fractional part is less than 0.5, round down.
2. If the fractional part is greater than 0.5, round up.
3. If the fractional part is exactly 0.5, choose the nearest even integer.

This rounding rule MUST be implemented using integer arithmetic. Implementations MUST NOT call platform floating-point rounding functions for canonical timestamp derivation.

For negative intermediate source timestamps, the builder MUST first apply the primary-presentation-start normalization offset in exact rational arithmetic, then convert the normalized non-negative timestamp to integer microseconds. The final pts_us values stored in SVP Core MUST be non-negative.

The manifest timebase object SHOULD record the rounding contract:

```
{
  "unit": "microseconds",
  "origin": "primary_presentation_start",
  "source_timebase_mode": "exact_rational",
  "rounding": "round_half_to_even"
}
```

8. ID model

Every record that can be referenced MUST have a stable ID.

ID prefixes:

```
media_       media source
vstream_     video stream
astream_     audio stream
wave_        waveform window
word_        transcript token
speaker_     anonymous speaker
speech_      speech region
speakerseg_  speaker segment
shot_        shot
scene_       scene
frame_       analysis frame
entity_      visual entity
track_       entity track
region_      spatial region
mask_        mask record
depth_       depth record
rel_         relationship
embed_       embedding record
embedset_    embedding set
label_       optional label
proc_        processor
model_       model artifact
```

IDs are lowercase ASCII strings. IDs MUST be unique within the package. IDs MUST NOT be reused for different objects across package rebuilds unless the object identity is stable and provenance records the reuse strategy.

8.1 Canonical model IDs

A canonical SVP model ID is an ID with the `model_` prefix. It identifies the model artifact as SVP knows it, independent of registry, hosting provider, filename, download URL, or source repository slug.

Examples:

```
model_depth_anything_v2_small
model_whisper_large_v3_turbo_q5_0
model_nomic_embed_text_v1_5
model_nomic_embed_vision_v1_5
```

The canonical model ID MUST be used for all normative references inside an SVP package or reference-builder artifact, including:

```
embedding_sets.json.model_id
provenance/model_hashes.jsonl.id
provenance/processors.jsonl.model_refs[]
DAG task model_refs[]
cache-key model_id inputs
model.svpmodel.json.model_id
model-lock.json references
validation and equivalence model identity fields
```

Provider-specific identifiers are source metadata only. A registry slug such as `nomic-ai/nomic-embed-text-v1.5` MAY be recorded as `source_slug`, but it MUST NOT be used as a normative `model_id`.

9. Manifest

`manifest.json` is the package entrypoint.

Required example:

```
{
  "svp_version": "1.0-rc.1",
  "package_id": "svp_01jz4a6r8y8y3v2v9jhd5k3x4g",
  "created_utc": "2026-06-18T00:00:00Z",
  "primary_media_id": "media_0001",
  "timebase": {
    "unit": "microseconds",
    "origin": "primary_presentation_start",
    "source_timebase_mode": "exact_rational",
    "rounding": "round_half_to_even"
  },
  "canonical_analysis_raster": {
    "width": 640,
    "height": 360,
    "derivation": "longest_display_dimension_640_preserve_dar",
    "source_display_width": 3840,
    "source_display_height": 2160,
    "source_display_aspect_ratio": "16:9",
    "rotation_applied": true,
```

```
    "pixel_aspect_ratio_applied": true,
    "coordinate_origin": "top_left",
    "normalized_coordinates": true
  },
  "binary_block_format": {
    "magic": "SVPB",
    "version": 1,
    "header_size": 160,
    "compression": "zstd"
  },
  "hashes": {
    "algorithm": "blake3",
    "digest_bytes": 32,
    "encoding": "lowercase_hex",
    "manifest_excluded": true
  },
  "required_sections": {
    "media": true,
    "transcript": true,
    "timeline": true,
    "entities": true,
    "spatial": true,
    "relationships": true,
    "embeddings": true,
    "index": true,
    "provenance": true
  }
}
```

RC1 defines package equivalence in Section 5.16. A manifest MAY include an `equivalence_policy` advisory object for reader discoverability, but it is not a conformance knob. If present, it MUST identify `svp-default-equivalence-v1` and MUST NOT declare looser thresholds than Section 5.16. If absent, Section 5.16 still applies.

The `canonical_analysis_raster` object MUST reflect the computed display-aspect-preserving analysis raster defined in Section 12.1. All spatial block headers and index records for masks and depth MUST match these dimensions exactly unless a block type explicitly defines non-image extents, such as embeddings.

The manifest MUST NOT contain detected object names, human identities, product names, or creative descriptions unless they are part of provenance. Labels belong in `/labels`.

10. Media section

10.1 Original video

`media/original/source_000.ext` MUST contain the original input media or a bit-exact copy. The file extension

SHOULD preserve the original extension.

The package MUST NOT require the original media to exist outside the SVP file.

10.2 Original audio

If the source contains audio, each original audio stream MUST be extracted into `media/audio/original_stream_NNN.flac` as lossless FLAC.

If the source does not contain audio, `original_stream_000.flac` MUST contain a canonical silent FLAC asset matching the source duration. `audio_absence.json` MUST state that the source had no audio.

10.3 Analysis audio

`media/audio/analysis_mono_16k.wav` MUST be present. It is a derived mono 16 kHz PCM WAV used for VAD, transcription, and diarization.

Primary audio selection rule:

1. If the source has one audio stream, use that stream.
2. If the source has multiple audio streams, use the first stream with speech detected by VAD.
3. If multiple streams contain speech, mix all speech-positive streams at equal gain.
4. If the source has no audio, create canonical silence.
5. Record the decision in `provenance/build.json`.

10.4 Waveform

`waveform.jsonl` stores audio envelope records in 10 ms windows:

```
{
  "id": "wave_00 000 001",
  "start_us": 0,
  "end_us": 10 000,
  "rms_db": −32.5,
  "peak_db": −18.1
}
```

Waveform data is required so apps and agents can reason about silence, loudness, spikes, and pacing without decoding audio.

11. Transcript section

11.1 Speech regions

`speech_regions.jsonl` contains VAD-positive speech intervals:

```
{
  "id": "speech_000 042",
  "start_us": 84 120 000,
  "end_us": 91 200 000,
  "start_sec": "84.120",
  "end_sec": "91.200",
  "confidence": 0.94,
  "processor_id": "proc_silero_vad_0001"
}
```


Silent media still contains the file. It may contain zero records.

11.2 Transcript summary

transcript.json contains document-level transcript metadata. The language field is a stable object, not a scalar string. Readers MUST NOT need to branch on array-or-string polymorphism.


```
{
  "language": {
    "primary": "en",
    "detected": ["en"],
    "mode": "single",
    "confidence": 0.94
  },
  "duration_us": 3 625 100 000,
  "word_count": 8421,
  "speaker_count": 2,
  "source_audio_id": "astream_analysis_0001",
  "processor_id": "proc_whispercpp_0001"
}
```


Language object rules:

1. language.primary MUST be an ISO 639 language code when a primary language is known.
2. language.detected MUST be an array of zero or more detected language codes ordered by confidence descending.
3. language.mode MUST be one of single, mixed, undetermined, or none.
4. language.confidence MUST be a number from 0.0 to 1.0.
5. ISO 639-1 alpha-2 codes SHOULD be used when available.
6. ISO 639-2 or ISO 639-3 alpha-3 codes MAY be used when no alpha-2 code exists.
7. und MUST be used for undetermined linguistic content.
8. zxx MUST be used for no linguistic content.
9. Word-level language detection is not part of SVP v1.0 Core. words.jsonl MUST NOT include a core language field. Word-level language annotations MAY appear only as non-core labels or in a future major version.

Mixed-language example:

```
{
  "language": {
    "primary": "en",
    "detected": ["en", "es"],
    "mode": "mixed",
    "confidence": 0.82
  },
  "duration_us": 3 625 100 000,
  "word_count": 8421,
  "speaker_count": 2,
  "source_audio_id": "astream_analysis_0001",
  "processor_id": "proc_whispercpp_0001"
}
```

If VAD detects speech-like regions but transcription yields no coherent conforming words, the core package remains valid with `word_count: 0`, an empty `words.jsonl`, retained `speech_regions.jsonl`, and `language.primary: "und"` with `language.mode: "undetermined"`.

```
{
  "language": {
    "primary": "und",
    "detected": [],
    "mode": "undetermined",
    "confidence": 0.0
  },
  "duration_us": 180 000 000,
  "word_count": 0,
  "speaker_count": 0,
  "source_audio_id": "astream_analysis_0001",
  "processor_id": "proc_whispercpp_0001"
}
```

If the source contains no linguistic content, such as music-only media or canonical silence, `language.primary` MUST be `zxx`, `language.mode` MUST be `none`, `word_count` MUST be `0`, and `speaker_count` MUST be `0`.

11.3 Word timestamps

`words.jsonl` is required and stores one word or punctuation token per line.

```
{
  "id": "word_000 538",
  "text": "camera",
  "normalized_text": "camera",
  "start_us": 84 120 000,
```

```
  "end_us": 84 490 000,
  "start_sec": "84.120",
  "end_sec": "84.490",
  "speaker_id": "speaker_0001",
  "speech_region_id": "speech_000 042",
  "confidence": 0.87,
  "speech_overlap": true,
  "speaker_candidates": [
   {
    "speaker_id": "speaker_0001",
    "confidence": 0.72
   },
   {
    "speaker_id": "speaker_0002",
    "confidence": 0.41
   }
  ]
}
```

Rules:

1. Words MUST be ordered by `start_us`.
2. Every word MUST have one primary `speaker_id` after diarization assignment.
3. `speaker_id` is singular for core compliance. Simple readers MUST be able to treat each word as having one primary speaker.
4. `speaker_candidates` is optional. When present, it MUST be an array of candidate speakers ordered by descending confidence. Each entry MUST contain `speaker_id` and `confidence`.
5. `speaker_candidates` MAY include the primary `speaker_id` and MAY include additional speakers detected during overlapping speech.
6. `speech_overlap` is optional. When present and true, it means the word occurs during a region where the diarization processor detected simultaneous speech or meaningful speaker collision.
7. Uncertainty and overlap are not the same. Low confidence without `speech_overlap: true` means uncertain attribution. `speech_overlap: true` means more than one speaker may be active during the word interval.
8. Punctuation tokens MAY have zero duration only if attached to the previous word through `attached_to_word_id`.
9. Filler words are stored as words. They MUST NOT be silently deleted.
10. The transcript is observational, not editorial.
11. Transcript correction is a label or revision layer in a future major version. It MUST NOT overwrite observed output in v1.0.

Overlapping speech does not weaken the core rule. Every word still has exactly one primary `speaker_id`. The optional candidate list preserves evidence that would otherwise be collapsed into one guess.

Word-level language spans are intentionally outside SVP v1.0 Core. The document-level `language` object in

transcript.json is the v1.0 mechanism for single-language, mixed-language, undetermined, and no-linguistic-content cases. A word record that contains an unrecognized language, language_code, or language_confidence field is not valid against the v1.0 Core word schema unless that field is carried through a future schema revision or non-core label mechanism.

11.4 Speakers

speakers.jsonl contains anonymous speaker records:

```
{
  "id": "speaker_0001",
  "display_name": "Speaker 1",
  "embedding_ref": "embed_speaker_0001",
  "total_speech_us": 2 814 400 000,
  "confidence": 0.91,
  "processor_id": "proc_sherpa_diar_0001"
}
```

display_name MUST be generic unless the labels section maps the speaker to a real identity.

11.5 Speaker segments

speaker_segments.jsonl stores diarization segments:

```
{
  "id": "speakerseg_000 122",
  "speaker_id": "speaker_0001",
  "start_us": 84 120 000,
  "end_us": 91 200 000,
  "start_sec": "84.120",
  "end_sec": "91.200",
  "confidence": 0.89,
  "overlap": false
}
```

12. Timeline section

12.1 Frames and canonical analysis raster

timeline/frames.jsonl maps decoded video frames to source presentation timestamps.

SVP v1.0 uses an aspect-preserving canonical analysis raster. The reference builder MUST compute it from displayed geometry, not raw stored pixel dimensions alone.

Canonical raster rule:

1. Apply source rotation metadata so the analysis raster matches intended display orientation.
2. Apply pixel aspect ratio to compute display aspect ratio.
3. Map the longest displayed dimension to `640` pixels.
4. Compute the shorter dimension from the display aspect ratio.
5. Round the shorter dimension to the nearest even integer. If two even integers are equally near, choose the larger even integer. Odd canonical analysis dimensions are forbidden.
6. Record the computed width and height in `manifest.json` under `canonical_analysis_raster`.
7. Both canonical analysis raster dimensions MUST be positive even integers. Every mask and depth image extent MUST match this computed canonical analysis raster exactly.

Examples:

Displayed source	Display aspect ratio	Canonical analysis raster
1920x1080 landscape	16:9	640x360
1080x1920 vertical	9:16	360x640
1080x1080 square	1:1	640x640
3840x2160 landscape	16:9	640x360
2048x858 scope	about 2.39:1	640x268

The spec intentionally forbids odd canonical analysis dimensions. This keeps tensor extents, mask packing, SIMD alignment, and fixture outputs deterministic across platforms. Implementations MUST NOT keep an odd shorter dimension merely because the unrounded display-ratio calculation produced an exact odd integer.

Every decoded source frame MUST have a frame record. Spatial outputs may be stored at the canonical raster, but every record must map back to source dimensions and displayed dimensions.


```
{
  "id": "frame_00 010 234",
  "frame_index": 10 234,
  "source_stream_id": "vstream_0001",
  "pts_us": 341 133 333,
  "pts_sec": "341.133333",
  "source_width": 3840,
  "source_height": 2160,
  "display_width": 3840,
  "display_height": 2160,
```

```
  "pixel_aspect_ratio": "1:1",
  "rotation_degrees_applied": 0,
  "analysis_width": 640,
  "analysis_height": 360,
  "shot_id": "shot_000 301",
  "scene_id": "scene_000 044"
}
```

12.2 Shots

A shot is a continuous interval of video between visual cuts, fades, or transitions.

timeline/shots.jsonl:

```
{
  "id": "shot_000 301",
  "start_us": 340 900 000,
  "end_us": 352 240 000,
  "start_frame_id": "frame_00 010 227",
  "end_frame_id": "frame_00 010 567",
  "cut_type_in": "hard_cut",
  "cut_type_out": "hard_cut",
  "visual_change_score": 42.8,
  "dominant_motion": "static",
  "black_frame_ratio": 0.0,
  "processor_id": "proc_shotdet_0001"
}
```

Controlled cut_type_in and cut_type_out values:

```
hard_cut
fade
cross_dissolve
black
unknown_transition
source_start
source_end
```

12.3 Scenes

A scene is a higher-level grouping of shots. It is still an observation layer, not a creative chapter label.

timeline/scenes.jsonl:

```
{
  "id": "scene_000 044",
```

```
  "start_us": 322 100 000,
  "end_us": 381 500 000,
  "shot_ids": ["shot_000 287", "shot_000 288", "shot_000 301"],
  "visual_embedding_ref": "embed_scene_vis_000 044",
  "text_embedding_ref": "embed_scene_text_000 044",
  "audio_continuity_score": 0.86,
  "visual_continuity_score": 0.78,
  "transcript_topic_shift_score": 0.22,
  "processor_id": "proc_scenegrp_0001"
}
```

13. Entity section

13.1 Entity definition

An SVP entity is a persistent visual thing or coherent visual region over time. Entity does not mean labeled object.

Examples:

```
A talking head
A phone-shaped region
A hand region
A screen region
A static background panel
A moving car-shaped region
```

A conforming SVP package does not need to know what any entity is called.

13.2 Entities

entities/entities.jsonl:

```
{
  "id": "entity_000 084",
  "entity_type": "visual_entity",
  "first_seen_us": 84 120 000,
  "last_seen_us": 129 020 000,
  "track_ids": ["track_000 084_a"],
  "primary_embedding_ref": "embed_entity_000 084",
  "average_visibility": 0.74,
  "average_screen_area": 0.183,
  "processor_id": "proc_entitytrack_0001"
}
```

entity_type values in v1.0:

```
visual_entity
background_region
screen_region
face_region
unknown_region
```

These are structural types, not labels. face_region means face-shaped detector output, not identity.

A video with no persistent visual entity may have zero entity records. Builders MUST NOT synthesize a fake background entity solely to satisfy non-empty output expectations. Empty visual entity output is valid when it is the honest observation result, and Section 13.4 defines the exact constraints.

13.3 Entity tracks

entities/entity_tracks.jsonl:

```
{
  "id": "track_000 084_a",
  "entity_id": "entity_000 084",
  "start_us": 84 120 000,
  "end_us": 129 020 000,
  "start_frame_id": "frame_00 002 523",
  "end_frame_id": "frame_00 003 870",
  "region_count": 1348,
  "lost_frame_count": 22,
  "reacquired": true,
  "confidence": 0.82,
  "processor_id": "proc_entitytrack_0001"
}
```

13.4 Degenerate visual sources

Some valid videos contain no persistent moving visual entity that the core tracker can honestly represent. Examples include:

```
static title card
slate
single still image encoded as video
near-zero-motion talking-head frame with no separable tracked region
static screen recording
black video
white video
solid-color placeholder
```

A conformant SVP package MUST still contain the required entities/, spatial/, depth, masks, relationships/, embeddings/, and index/ files for these sources. Strict mode MUST NOT require the builder to invent entity tracks when no persistent visual entity is observed.

For degenerate visual sources:

1. entities/entities.jsonl MAY contain zero records.
2. entities/entity_tracks.jsonl MAY contain zero records.
3. spatial/regions.jsonl MAY contain zero records.
4. spatial/masks.index.jsonl MAY contain zero records.
5. spatial/masks.blocks.svpmz MAY be a zero-length block stream only when masks.index.jsonl contains zero records and no region references a mask.
6. relationships/relationships.jsonl MAY contain no visual relationships.
7. spatial/depth.index.jsonl and spatial/depth.blocks.svpdz MUST still represent the analyzed video frame ranges. Near-uniform depth output is valid when the source is visually uniform or depth cannot vary meaningfully.
8. The SQLite index MUST still contain the required tables. Tables representing entities, regions, masks, and visual relationships MAY contain zero rows.

Empty tracks are valid only when they are the honest observation result. They are not a loophole for skipping visual processing.

14. Spatial section

14.1 Coordinate system

SVP uses two coordinate systems:

1. Normalized coordinates in [0.0, 1.0] relative to the visible displayed frame.
2. Analysis raster pixel coordinates relative to the computed canonical analysis raster defined in Section 12.1.

Origin is top-left. x increases right. y increases down.

The canonical analysis raster is package-specific. A 16:9 landscape package usually uses 640x360, a vertical package usually uses 360x640, and a square package uses 640x640. Spatial readers MUST read manifest.json and MUST NOT assume fixed 640x360 dimensions.

For image block types, SVPBlockHeaderV1.extent_0 is width and extent_1 is height. For depth and mask blocks, those extents MUST match manifest.canonical_analysis_raster.width and manifest.canonical_analysis_raster.height exactly.

14.2 Regions

A region is an entity's spatial observation at a specific frame.

spatial/regions.jsonl:

```
{
  "id": "region_000 084_00 010 234",
  "entity_id": "entity_000 084",
  "track_id": "track_000 084_a",
  "frame_id": "frame_00 010 234",
  "pts_us": 341 133 333,
  "box_norm": [0.4125, 0.1833, 0.7468, 0.9222],
  "box_px": [264, 66, 478, 332],
  "centroid_norm": [0.5796, 0.5527],
  "screen_area_ratio": 0.247,
  "mask_ref": "mask_000 084_00 010 234",
  "depth_ref": "depth_00 010 234",
  "depth_summary": {
    "median_inverse_depth": 0.712,
    "near_percentile_10": 0.812,
    "far_percentile_90": 0.424
  },
  "visibility": 0.91,
  "occlusion_ratio": 0.04,
  "confidence": 0.86
}
```


box_norm and box_px are both required.

14.3 Masks

Masks are stored as compressed binary foreground observations at the computed canonical analysis raster. RC1 stores masks in an SVP mask block stream, not in a raw .zst file. If no regions exist, the mask index may be empty and the mask block stream may be zero bytes, as constrained by Section 13.4 and the validation rules.

spatial/masks.index.jsonl:


```
{
  "id": "mask_000 084_00 010 234",
  "region_id": "region_000 084_00 010 234",
  "frame_id": "frame_00 010 234",
  "width": 640,
  "height": 360,
  "encoding": "svp-rle-v1",
  "block_file": "spatial/masks.blocks.svpmz",
  "block_offset": 8 821 044,
  "block_length": 576,
  "payload_offset": 160,
  "uncompressed_size": 416,
  "compressed_size": 396,
  "payload_blake3": "3cf2...",
```

```
  "block_blake3": "51db..."
}
```

RLE rules:

1. Masks are binary foreground masks.
2. RLE starts with background run length.
3. Runs are unsigned LEB128 integers.
4. Scan order is row-major from top-left to bottom-right.
5. The decoded run total MUST equal `width * height`. The `width` and `height` values MUST match the package canonical analysis raster.

14.4 Depth

Depth is stored per frame at the computed canonical analysis raster as normalized relative inverse depth. RC1 stores depth in an SVP depth block stream, not in a raw `.zst` file. Near-uniform depth is valid for static, flat, blank, or otherwise degenerate visual sources when provenance records successful depth processing.

`spatial/depth.index.jsonl`:

```
{
  "id": "depth_00 010 234",
  "frame_id": "frame_00 010 234",
  "width": 640,
  "height": 360,
  "value_type": "uint16_relative_inverse_depth",
  "normalization": "near_is_larger",
  "block_file": "spatial/depth.blocks.svpdz",
  "block_offset": 18 821 440,
  "block_length": 195 832,
  "payload_offset": 160,
  "uncompressed_size": 460 800,
  "compressed_size": 195 672,
  "processor_id": "proc_depth_0001",
  "payload_blake3": "91af...",
  "block_blake3": "04d9..."
}
```

Depth values:

```
0      = farthest valid relative depth in the frame
65 535  = nearest valid relative depth in the frame
```

Depth block `width` and `height` values MUST match the package canonical analysis raster. Depth is relative inside the observed frame or shot context. Cross-shot absolute comparison MUST NOT be assumed in

v1.0.

14.5 SVP binary block header

Depth, mask, and embedding payload files are SVP block streams. A block stream is a concatenation of one or more blocks:

```
[SVPBlockHeaderV1][compressed payload][SVPBlockHeaderV1][compressed payload]...
```

Every block header is exactly 160 bytes in RC1. All integer fields are little-endian. Readers MUST reject blocks with unknown `version`, unsupported `compression`, invalid `header_size`, mismatched hashes, invalid block type for strict v1.0, or payload ranges outside the declared file length.

Header layout:

Offset	Size	Field	Type	Required value or meaning
0	4	magic	bytes	`SVPB`
4	2	version	uint16	`1`
6	2	header_size	uint16	`160`
8	1	block_type	uint8	`0x01` depth, `0x02` mask, `0x03` embedding. `0x04` is reserved for future audio feature tensors and MUST NOT appear in strict v1.0 packages.
9	1	compression	uint8	`0x01` Zstandard
10	1	endian	uint8	`0x01` little-endian
11	1	flags	uint8	`0` in RC1
12	8	uncompressed_size	uint64	Decompressed payload byte count
20	8	compressed_size	uint64	Compressed payload byte

Offset	Size	Field	Type	Required value or meaning
				count
28	4	extent_0	uint32	Width for image blocks, vector count for embedding blocks
32	4	extent_1	uint32	Height for image blocks, vector dimension for embedding blocks
36	4	extent_2	uint32	Channel or plane count
40	4	dtype	uint32	Payload dtype enum
44	8	start_frame	uint64	First frame index, or `UINT64_MAX` for non-frame blocks
52	8	frame_count	uint64	Number of frames represented, or `0` for non-frame blocks
60	8	start_us	int64	Inclusive start time, or `-1` when not time-bound
68	8	end_us	int64	Exclusive end time, or `-1` when not time-bound
76	32	payload_blake3	bytes	BLAKE3 of

Offset	Size	Field	Type	Required value or meaning
				compressed payload bytes
108	32	header_blake3	bytes	BLAKE3 of header bytes with this field zeroed
140	20	reserved	bytes	All zero

`header_blake3` is computed over the 160-byte header after replacing bytes `108..139` with zero bytes. `payload_blake3` is computed over the compressed payload exactly as stored after the header.

Dtype enum:

Value	Meaning
1	uint8
2	uint16
3	float16
4	float32
5	bitpacked_lsb_first
6	svp-rle-v1

Block type registry:

Value	Name	Strict v1.0 package behavior
0x00	Invalid	Forbidden. Readers MUST reject.
0x01	Depth tensor	Required for depth blocks.
0x02	Mask tensor	Required for mask blocks.
0x03	Embedding tensor	Required for embedding blocks.

Value	Name	Strict v1.0 package behavior
0x04	Audio feature tensor	Reserved for a future SVP version. MUST NOT appear in strict v1.0 packages.
0x05–0x7F	Future SVP core block types	Reserved. MUST NOT appear in strict v1.0 packages.
0x80–0xFE	Implementation-private experimental	Forbidden in strict v1.0 packages. Experimental streams MUST NOT be placed inside .svp Core.
0xFF	Sentinel	Forbidden as a persisted block type.

Block type conventions:

```
Depth block:
  block_type = 0x01
  extent_0 = width
  extent_1 = height
  extent_2 = 1
  dtype = 2 for uint16 relative inverse depth

Mask block:
  block_type = 0x02
  extent_0 = width
  extent_1 = height
  extent_2 = mask planes in the block
  dtype = 5 for bitpacked masks or 6 for svp-rle-v1

Embedding block:
  block_type = 0x03
  extent_0 = vector count
  extent_1 = vector dimension
  extent_2 = 1
  dtype = 4 for float32
```

Compressed blocks are not zero-copy to decoded arrays. The correct performance guarantee is memory-mappable block discovery with bounded decompression.

15. Relationship graph

Relationships are required. They turn disconnected observations into queryable structure.

relationships/relationships.jsonl:

```
{
  "id": "rel_00 044 210",
  "type": "overlaps",
  "source_id": "entity_000 084",
  "target_id": "entity_000 012",
  "start_us": 341 133 333,
  "end_us": 343 100 000,
  "evidence": {
    "region_ids": ["region_000 084_00 010 234", "region_000 012_00 010 234"],
    "iou": 0.17,
    "depth_order": "source_in_front"
  },
  "confidence": 0.79,
  "processor_id": "proc_relationships_0001"
}
```

Controlled relationship types in v1.0:

```
appears_in_shot
appears_in_scene
visible_during_speech
visible_during_word_range
overlaps
near
contains
contained_by
enters_frame
exits_frame
occludes
occluded_by
moves_with
stationary_relative_to_camera
foreground_relative_to
background_relative_to
speaker_active_during_entity_visible
```

Relationships are observations computed from time, masks, depth, transcript, speaker segments, and motion. They are not creative descriptions.

16. Embeddings

16.1 Embedding sets

embeddings/embedding_sets.json describes all embedding spaces.

```
{
  "sets": [
    {
      "id": "embedset_text_nomic_v15",
      "modality": "text",
      "model_id": "model_nomic_embed_text_v1_5",
      "model_blake3": "...",
      "dimension": 768,
      "dtype": "float32",
      "normalized": true,
      "source_slug": "nomic-ai/nomic-embed-text-v1.5"
    },
    {
      "id": "embedset_vision_nomic_v15",
      "modality": "vision",
      "model_id": "model_nomic_embed_vision_v1_5",
      "model_blake3": "...",
      "dimension": 768,
      "dtype": "float32",
      "normalized": true,
      "source_slug": "nomic-ai/nomic-embed-vision-v1.5"
    }
  ]
}
```

16.2 Embedding index

embeddings/embeddings.index.jsonl:

```
{
  "id": "embed_scene_text_000 044",
  "embedding_set_id": "embedset_text_nomic_v15",
  "input_ref": "scene_000 044",
  "input_kind": "scene_transcript_window",
  "start_us": 322 100 000,
  "end_us": 381 500 000,
  "block_file": "embeddings/embeddings.blocks.svpez",
  "block_offset": 1 048 576,
  "block_length": 98 432,
  "vector_index": 42,
```

```
  "dimension": 768,
  "dtype": "float32",
  "payload_blake3": "b8a1...",
  "block_blake3": "74bc..."
}
```

Embedding blocks use the SVP binary block header with `block_type = 0x03`, `extent_0 = vector count`, `extent_1 = vector dimension`, `extent_2 = 1`, and `dtype = 4`.

Required embedding coverage:

```
Every transcript chunk
Every scene
Every shot keyframe
Every entity primary crop
Every speaker voice embedding
Every relationship text projection
```

Transcript chunking rule:

1. Chunks target 256 words.
2. Chunks overlap by 32 words.
3. Chunk boundaries MUST align to word IDs.
4. Chunk records MUST be written to the search index.

16A. OCR and visible-text observations

SVP RC2 makes visible text and numbers first-class observations. The package MUST contain `/text/` for any SVP package with video media.

Observed visible text is not a label. The string `SALE $9.99` visible in a frame is an observation. Interpreting the frame as an advertisement is a label or higher-level interpretation.

16A.1 Required text files

```
text/
  text_regions.jsonl
  text_observations.jsonl
  numeric_values.jsonl
  text_absence.json
```

Rules:

1. `/text/text_regions.jsonl` MUST exist. It MAY be empty only when the OCR processor completed and

found no text regions.

2. /text/text_observations.jsonl MUST exist. It MAY be empty only when there are no recognized text observations.
3. /text/numeric_values.jsonl MUST exist. It MAY be empty when no safely parseable numeric values are observed.
4. /text/text_absence.json MUST exist and MUST state whether no text was detected, OCR was skipped due to unsupported media, or OCR completed with zero results. For normal video, OCR MUST NOT be skipped by a conforming builder.
5. Every text record MUST map to time and SHOULD reference the most specific applicable target: frame, shot, scene, region, entity, or text region.

16A.2 Text region record

A text region is a detected region of the canonical analysis raster that likely contains visible text.


```
{
  "text_region_id": "text_region_000 001",
  "observation_type": "text_detection",
  "start_us": 1 200 000,
  "end_us": 2 400 000,
  "frame_start": 36,
  "frame_end": 72,
  "shot_id": "shot_000 003",
  "scene_id": "scene_000 001",
  "region_id": "region_000 912",
  "entity_id": "entity_000 044",
  "bbox_norm": [0.4125, 0.2014, 0.6812, 0.2528],
  "bbox_px": [264, 73, 436, 91],
  "orientation_deg": 0.0,
  "text_direction": "ltr",
  "script": "Latn",
  "confidence": 0.944,
  "foreground_color_observation_id": "color_obs_000 331",
  "background_color_observation_id": "color_obs_000 332",
  "contrast_ratio": 7.4,
  "legibility_score": 0.88,
  "provenance_id": "processor_ocr_detector_0001"
}
```


bbox_norm is [x_min, y_min, x_max, y_max] in normalized coordinates over the canonical analysis raster. bbox_px is the same box in canonical analysis raster pixels.

16A.3 Text observation record

A text observation is the recognition result attached to a text region.

```
{
  "text_observation_id": "text_obs_000 001",
  "text_region_id": "text_region_000 001",
  "observation_type": "text_recognition",
  "raw_text": "SALE $9.99",
  "normalized_text": "sale 9.99",
  "language": {
    "primary": "en",
    "script": "Latn",
    "mode": "single",
    "confidence": 0.81
  },
  "confidence": 0.902,
  "layout_class": "ui_text",
  "source_frame_ids": ["frame_000 036", "frame_000 048", "frame_000 060"],
  "provenance_id": "processor_ocr_recognizer_0001"
}
```


raw_text stores what the recognizer returned. normalized_text stores the canonical query string after Unicode normalization, case folding where applicable, whitespace normalization, and removal of purely decorative control characters. Normalization MUST NOT erase the raw observed string.

16A.4 Numeric value record

A numeric value record exists only when the builder can safely parse a number from a text observation without inventing semantics.


```
{
  "numeric_value_id": "numeric_value_000 001",
  "text_observation_id": "text_obs_000 001",
  "text_region_id": "text_region_000 001",
  "raw_text": "$9.99",
  "normalized_text": "9.99",
  "number_kind": "decimal",
  "numeric_value": "9.99",
  "unit": "currency_unknown",
  "confidence": 0.89,
  "parse_rule": "svp-number-parser-v1",
  "provenance_id": "processor_numeric_parser_0001"
}
```


Numeric values are stored as decimal strings, not binary floating-point values. Currency symbols, counters, timestamps, chart labels, and UI values MUST preserve the raw text and MUST NOT be interpreted beyond safe parsing.

16A.5 OCR observation type registry

The following observation types are registered in /spec/ocr-observation-types.json:

```
text_detection
text_recognition
number_extraction
layout_analysis
subtitle_caption
ui_text
scene_text_signage
chart_graph_number
timestamp_counter
```

A strict v1.0 package MUST NOT use unregistered OCR observation types in Core records.

16A.6 OCR absence

text_absence.json records completed OCR state:

```
{
  "schema_version": "svp-text-absence-v1",
  "ocr_required": true,
  "ocr_completed": true,
  "text_region_count": 0,
  "text_observation_count": 0,
  "numeric_value_count": 0,
  "reason": "no_text_detected",
  "provenance_id": "processor_ocr_detector_0001"
}
```

A package with no visible text is still Core-valid if the required text files exist, the absence record shows successful completion, and the index agrees with the text records.

16B. Structured color observations

SVP RC2 makes measurable color distribution a first-class observation layer. Color observations are required for video packages and MUST be stored in /colors/.

Color records describe sampled pixel distributions under a registered color space and registered bucket rules. They do not claim real-world material color. They are measurements of encoded pixels after the builder's defined color normalization pipeline.

16B.1 Required color files

```
colors/
  color_observations.jsonl
  color_summary.json
  color_absence.json
```

Rules:

1. /colors/color_observations.jsonl MUST exist.
2. /colors/color_summary.json MUST exist and MUST summarize counts, bucket registry version, color space, and sampling policies used.
3. /colors/color_absence.json MUST exist and MUST state whether color observation processing completed.
4. Color observations MUST be emitted for every scene and every shot.
5. Color observations SHOULD be emitted for frames/keyframes, regions, entities, and text regions when those targets exist.
6. Text-region foreground and background color observations SHOULD be emitted when OCR text regions exist and quality is sufficient.

16B.2 Color observation record


```
{
  "color_observation_id": "color_obs_000 001",
  "target_type": "shot",
  "target_id": "shot_000 003",
  "start_us": 1 200 000,
  "end_us": 2 400 000,
  "frame_ids": ["frame_000 036", "frame_000 048", "frame_000 060"],
  "sampling_basis": "keyframe_full_frame",
  "color_space": "svp_oklch_v1",
  "color_bucket_registry_version": "svp-color-buckets-v1",
  "bucket_coverage": {
    "orange": 0.7000,
    "yellow": 0.0300,
    "black": 0.1200,
    "white": 0.0800,
    "other": 0.0700
  },
  "dominant_bucket": "orange",
  "palette": [
    {"space": "srgb", "hex": "#f47a21", "coverage": 0.41},
    {"space": "srgb", "hex": "#121 212", "coverage": 0.10}
  ],
  "coverage_total": 1.0000,
```

```
  "quality_score": 0.96,
  "provenance_id": "processor_color_quantizer_0001"
}
```

16B.3 Target types

target_type MUST be one of:

```
scene
shot
frame
region
entity
text_region
```

The target_id MUST reference a valid record of the declared target type.

16B.4 Sampling basis

sampling_basis MUST be one of:

```
full_frame
keyframe_full_frame
mask
region_crop
entity_mask
text_box
text_foreground
text_background
```

The sampling basis determines which pixels contribute to the bucket distribution.

16B.5 Color bucket registry

The authoritative color bucket definitions live in /spec/color-buckets.json. The package records the registry version used by each observation. A strict v1.0 package MUST NOT use unregistered bucket IDs in Core color observations.

RC2 uses numeric bucket coverage ratios. A query such as orange > 0.5 means that more than half of sampled pixels fell into the registered orange bucket under the registered color-space and quantization rule.

16B.6 Color summary and absence

color_summary.json records package-level color processing state:

```
{
  "schema_version": "svp-color-summary-v1",
  "color_observation_count": 842,
  "color_space": "svp_oklch_v1",
  "color_bucket_registry_version": "svp-color-buckets-v1",
  "percentage_sum_tolerance": 0.001,
  "provenance_id": "processor_color_quantizer_0001"
}
```

color_absence.json records whether processing completed. Unlike text_absence.json, color observations should normally not be absent for video. If the source video has only black frames, white frames, or uniform cards, color observations are still valid and should report the measured distribution.

17. Search index

index/index.sqlite is mandatory. It is not a cache.

17.1 Required SQLite tables

The reference schema begins:

```
PRAGMA user_version = 100;

CREATE TABLE svp_meta (
  key TEXT PRIMARY KEY,
  value TEXT NOT NULL
);

CREATE TABLE objects (
  object_id TEXT PRIMARY KEY,
  object_type TEXT NOT NULL,
  start_us INTEGER,
  end_us INTEGER,
  json_path TEXT NOT NULL
);

CREATE TABLE temporal_spans (
  object_id TEXT NOT NULL,
  object_type TEXT NOT NULL,
  start_us INTEGER NOT NULL,
  end_us INTEGER NOT NULL,
  FOREIGN KEY(object_id) REFERENCES objects(object_id)
);

CREATE TABLE relationships (
  relationship_id TEXT PRIMARY KEY,
```

```
  relationship_type TEXT NOT NULL,
  source_id TEXT NOT NULL,
  target_id TEXT NOT NULL,
  start_us INTEGER NOT NULL,
  end_us INTEGER NOT NULL,
  confidence REAL NOT NULL
);

CREATE VIRTUAL TABLE text_fts USING fts5(
  object_id UNINDEXED,
  object_type UNINDEXED,
  start_us UNINDEXED,
  end_us UNINDEXED,
  text,
  tokenize = 'unicode61'
);
```

17.2 Required vector table

The reference processor uses sqlite-vec:

```
CREATE VIRTUAL TABLE vector_index USING vec0(
  embedding float[768],
  object_id TEXT auxiliary,
  object_type TEXT auxiliary,
  embedding_set_id TEXT auxiliary,
  start_us INTEGER auxiliary,
  end_us INTEGER auxiliary
);
```

17.3 Required binary block lookup table

The SQLite index MUST include a block lookup table so readers can resolve binary payloads without scanning block streams.

```
CREATE TABLE binary_blocks (
  block_id TEXT PRIMARY KEY,
  block_type TEXT NOT NULL,
  block_file TEXT NOT NULL,
  block_offset INTEGER NOT NULL,
  block_length INTEGER NOT NULL,
  payload_offset INTEGER NOT NULL,
  uncompressed_size INTEGER NOT NULL,
  compressed_size INTEGER NOT NULL,
  extent_0 INTEGER NOT NULL,
  extent_1 INTEGER NOT NULL,
```

```
  extent_2 INTEGER NOT NULL,
  dtype INTEGER NOT NULL,
  start_frame INTEGER,
  frame_count INTEGER,
  start_us INTEGER,
  end_us INTEGER,
  payload_blake3 TEXT NOT NULL,
  header_blake3 TEXT NOT NULL,
  block_blake3 TEXT NOT NULL
);

CREATE INDEX binary_blocks_file_offset
ON binary_blocks(block_file, block_offset);
```

binary_blocks.block_offset points to the first byte of an SVPBlockHeaderV1 header. binary_blocks.block_length includes the header and compressed payload.

17.4 Hybrid query expectation

Readers SHOULD support hybrid retrieval:

```
FTS5 keyword match
sqlite-vec vector similarity
temporal filters
relationship filters
```

The reference query API exposes this as one operation:

```
svp query video.svp \
  "foreground entity visible while Speaker 1 says camera"
```

The query command returns object IDs and time spans, not an edit decision.

17.5 SQLite logical reproducibility and source-layer-aware equivalence

index/index.sqlite is evaluated for logical reproducibility, not byte-for-byte file reproducibility, during package equivalence comparison.

SQLite database files can differ physically across otherwise equivalent builds because of page allocation, row insertion history, free lists, vacuum state, journal history, and internal storage layout. These details are not SVP package-equivalence facts.

There are two related but distinct checks:

1. **Single-package logical integrity.** A validator checks one package against its own index/index_manifest.json. The validator produces the canonical logical row stream from the package's

actual `index/index.sqlite` contents and verifies `index_manifest.logical_rows_blake3`.

2. **Cross-package equivalence.** A validator compares two valid packages using source-layer-aware SQLite equivalence. It MUST NOT require exact logical-row hash equality for columns whose source values are governed by the Default Equivalence Profile.

The validator MUST generate the canonical logical row stream for single-package integrity as follows:

1. Open the database in read-only mode with extension loading disabled.
2. Enumerate all user-defined tables and virtual tables in schema-name order.
3. Exclude SQLite internal tables whose names begin with `sqlite_`, except where the SVP schema explicitly requires their logical contents.
4. Emit each table name.
5. Emit the normalized schema fingerprint for each table.
6. Emit column names in ordinal schema order.
7. Emit rows ordered by declared primary key.
8. If a table has no declared primary key, emit rows ordered by all columns in ordinal schema order using canonical scalar ordering.
9. Serialize values using SVP canonical JSON scalar rules.
10. Hash the resulting UTF-8 logical row stream with BLAKE3.

The raw bytes of `index/index.sqlite` MAY differ across equivalent builds. For single-package integrity, the canonical logical row-stream BLAKE3 MUST match the package's own `index/index_manifest.json` field `logical_rows_blake3`.

For cross-package equivalence, SQLite columns are assigned to one of three equivalence classes.

Exact SQLite values MUST match exactly:

```
table names
schema fingerprints
column names
object IDs
section names
schema versions
processor IDs
model IDs
embedding_set_id values
object_type values
source refs
JSON paths
time ranges
frame ranges
block_file
block_offset
block_length
payload_offset
uncompressed_size
```

```
compressed_size
extent_0, extent_1, extent_2
dtype
relationship rows
FTS text
FTS object mappings
all non-derived integer and string metadata
```

Numerical SQLite values MUST use the relevant Default Equivalence Profile rule:

```
vector_index.embedding
REAL confidence columns generated by floating-point processors
any future core numeric column explicitly declared tolerance-governed by its source layer
```

`vector_index.embedding` values MUST be compared with the same cosine similarity threshold used for embedding payloads in Section 5.16.2. Vector metadata such as `object_id`, `object_type`, `embedding_set_id`, `start_us`, and `end_us` remains exact-match.

Hash columns for tolerance-governed payloads are handled by source-layer equivalence, not by hash tolerance:

```
binary_blocks.payload_blake3
binary_blocks.header_blake3
binary_blocks.block_blake3
```

BLAKE3 hashes themselves are never tolerance-compared. When these columns reference depth, mask, embedding, vector, or other tolerance-governed derived payloads, an exact hash mismatch MUST NOT by itself cause `not_equivalent` if the referenced decoded payloads satisfy the applicable numerical equivalence rule in Section 5.16.2. The validator MUST report this condition as `hash_mismatch_allowed_by_source_layer_equivalence` or an equivalent machine-readable detail.

When these hash columns reference exact-governed bytes, such as original media, lossless audio, canonical JSON, canonical JSONL, schema fingerprints, or non-derived package artifacts, exact hash match remains required.

All columns not explicitly assigned to numerical or source-payload-hash handling are exact-match columns.

17.6 Index manifest

`index/index_manifest.json` is a required Core file. It records the logical integrity of `index/index.sqlite` and gives validators enough information to verify the SQLite index without comparing raw SQLite database bytes.

Required example:

```
{
  "schema_version": "svp-index-manifest-v1",
  "index_schema_version": "svp-index-v1",
  "sqlite_file": "index/index.sqlite",
  "sqlite_file_blake3": "blake3:...",
  "logical_row_stream_version": "svp-logical-row-stream-v1",
  "logical_rows_blake3": "blake3:...",
  "table_count": 12,
  "row_count": 184233,
  "created_from": {
    "manifest_blake3": "blake3:...",
    "binary_blocks_manifest_blake3": "blake3:...",
    "embedding_sets_blake3": "blake3:..."
  }
}
```

Required fields:

```
schema_version
index_schema_version
sqlite_file
sqlite_file_blake3
logical_row_stream_version
logical_rows_blake3
table_count
row_count
created_from
```

`schema_version` MUST be `svp-index-manifest-v1` for SVP v1.0.

`sqlite_file` MUST be `index/index.sqlite`.

`sqlite_file_blake3` is the exact BLAKE3 of the physical SQLite database file. It is an integrity fact for the package as built. It is not used for cross-package equivalence because physical SQLite bytes are not required to be reproducible.

`logical_row_stream_version` MUST identify the canonical logical row-stream procedure. For SVP v1.0 it MUST be `svp-logical-row-stream-v1`.

`logical_rows_blake3` MUST be the BLAKE3 of the canonical logical row stream generated by Section 17.5 steps 1 through 10 from the package's actual `index/index.sqlite` contents. This value is used for single-package logical integrity.

`table_count` and `row_count` record the number of user-defined logical tables and rows included in the canonical logical row stream. They are validator sanity checks and MUST match the stream used to compute `logical_rows_blake3`.

created_from MUST include BLAKE3 references to the package records that materially define the index. At minimum it MUST include manifest_blake3, binary_blocks_manifest_blake3, and embedding_sets_blake3. Builders MAY include additional exact BLAKE3 references when the schema defines them.

The required schema file is:

```
/schemas/index-manifest.schema.json
```

A strict validator MUST fail the package if index/index_manifest.json is absent, malformed, missing a required field, or if the generated Section 17.5 logical row stream does not match logical_rows_blake3.

17.7 Required OCR and visible-text index tables

The SQLite index MUST include tables that support visible text, normalized visible text, numeric values, and time/shot/scene lookups.

Required logical tables:

```
text_regions
text_observations
numeric_values
text_fts
```

Minimum query support:

```
visible text by raw_text
visible text by normalized_text
numeric values by exact decimal string
text by time range
text by scene_id
text by shot_id
text by region_id
text by entity_id
text by layout_class
```

Example query targets:

```
all scenes containing visible text "SALE"
all shots with numeric value "99"
all UI text regions containing green foreground text
all chart/graph numbers during a selected scene
```

17.8 Required color index tables

The SQLite index MUST include tables that support color bucket coverage queries by scene, shot, frame,

region, entity, and text region.

Required logical tables:

```
color_observations
color_bucket_coverage
color_targets
```

Minimum query support:

```
color bucket percentage by scene
color bucket percentage by shot
color bucket percentage by frame or keyframe
color bucket percentage by region
color bucket percentage by entity
color bucket percentage by text_region
foreground/background color lookup for text regions
```

Example query targets:

```
all scenes where orange coverage > 0.5
all shots where yellow coverage >= 0.03
all frames with red warning text
all text regions with red foreground and high contrast
all UI regions with green numbers
```

17.9 OCR/color index consistency

Records in `/text/` and `/colors/` MUST be reflected in `index/index.sqlite`. A validator MUST compare package records against the corresponding SQLite logical rows. A mismatch is a Core validation failure.

18. Provenance

18.0 OCR and color provenance requirements

OCR provenance MUST record processor ID, processor version, model ID when model-backed, model bundle ID, bundle BLAKE3, runtime, execution provider, detection thresholds, recognition thresholds, language/script assumptions, source frame references, and text postprocessing rules.

Color provenance MUST record processor ID, processor version, color space, color bucket registry version, quantization method, sampling basis, canonical raster basis, rounding and precision rules, target type, and quality metrics. Color processing may be deterministic classical processing and need not reference a model ID.

18.1 Processor and model provenance

Every processor writes a record to `provenance/processors.jsonl`.

```
{
  "id": "proc_whispercpp_0001",
  "name": "whisper.cpp",
  "version": "1.8.6",
  "command": "whisper-cli -m model.bin -f analysis.wav -ml 1",
  "input_refs": ["media/audio/analysis_mono_16k.wav"],
  "output_refs": ["transcript/words.jsonl"],
  "model_refs": ["model_whisper_large_v3_turbo_q5_0"],
  "task_ids": ["task.transcript.audio_chunk_000 014"],
  "cache_keys": ["b3:..."],
  "started_utc": "2026-06-18T00:00:00Z",
  "completed_utc": "2026-06-18T00:00:00Z"
}
```

Model hash records:

```
{
  "id": "model_depth_anything_v2_small",
  "model_bundle_id": "model_depth_anything_v2_small@2024-06-13+blake3_8f4c21aa93d0",
  "name": "Depth Anything V2 Small",
  "source_registry": "github",
  "source_slug": "DepthAnything/Depth-Anything-V2",
  "source_revision": "2024-06-13-pinned",
  "license": "Apache-2.0",
  "blake3": "...",
  "bundle_blake3": "...",
  "alternate_hashes": {
    "sha256": "..."
  }
}
```

Processor records MUST identify the exact input references, output references, processor version, canonical model references, model bundle IDs, parameter set, runtime, execution provider, and task IDs used to generate each artifact.

18.2 Validation report

`provenance/validation.json` is the machine-readable validation report emitted by the reference builder after final package validation. It is also the required report schema for `svp validate --strict --report` and `svp validate --equivalent --report`.

A completed reference-builder package MUST include `provenance/validation.json`. The validation report is

provenance about conformance; it is not an observation layer and it MUST NOT be used to override core records.

RC1 requires /spec/validation-codes.json as a normative machine-readable registry published alongside the JSON schemas. Validators MUST use registered standard codes for standard conditions. Implementation-specific codes MAY exist only under an implementation namespace beginning with X_, and MUST NOT redefine the meaning of standard ERR_, WARN_, or INFO_ codes.

18.2.1 Core status and authenticity status

Validation reports have separate Core and authenticity result channels.

status and validator default exit code are computed only from Core validation findings. A Core-valid package remains Core-valid even when a provided detached signature is missing, unreadable, untrusted, or mismatched.

authenticity_status is computed only from detached-signature verification when authenticity verification is requested. Authenticity findings MUST be placed in the authenticity array. They MAY have severity error, but they are excluded from Core status computation and the default validator exit code.

A validator MAY provide an explicit signature-enforcement mode, such as svp validate --require-valid-signature. In that mode, signature mismatch MAY produce a non-zero process exit for the command. This is an invocation policy and MUST NOT redefine SVP Core validity.

Example strict-validation report with separate authenticity findings:


```
{
 "schema_version": "1.0-rc.1",
 "validator": {
  "name": "svp-validator",
  "version": "1.0.0-rc.1"
 },
 "status": "valid_with_warnings",
 "core_status": "valid_with_warnings",
 "authenticity_status": "signature_mismatch",
 "checked_utc": "2026-06-19T00:00:00Z",
 "package_hash": "blake3:...",
 "errors": [],
 "warnings": [
  {
   "code": "WARN_UNSUPPORTED_LABEL_SCHEMA",
   "severity": "warning",
   "path": "/labels/labels.jsonl",
   "message": "Labels schema version is newer than this validator supports. Core validation is unaffected."
  }
 ],
 "infos": [
  {
```

```
      "code": "INFO_DEGENERATE_VISUAL_SOURCE",
      "severity": "info",
      "path": "/entities/entity_tracks.jsonl",
      "message": "No persistent visual entity tracks were observed. Required visual sections are present and valid."
    }
  ],
  "authenticity": [
    {
      "code": "ERR_SIGNATURE_MISMATCH",
      "severity": "error",
      "path": "video.svpsig",
      "message": "Detached signature did not verify against the exact finalized .svp package bytes. Core validation is unaffected."
    }
  ]
}
```

Example equivalence-validation metrics block:

```
{
  "schema_version": "1.0-rc.1",
  "validator": {
    "name": "svp-validator",
    "version": "1.0.0-rc.1"
  },
  "status": "numerically_equivalent",
  "checked_utc": "2026-06-19T00:00:00Z",
  "packages": [
    {
      "path": "cpu-build.svp",
      "package_hash": "blake3:..."
    },
    {
      "path": "coreml-build.svp",
      "package_hash": "blake3:..."
    }
  ],
  "metrics": {
    "depth": {
      "status": "passed_within_tolerance",
      "rmse": 0.0021,
      "threshold_rmse": 0.005,
      "max_absolute_error": 0.0094
    },
    "embeddings": {
      "status": "passed_within_tolerance",
      "minimum_cosine_similarity": 0.99921,
      "threshold": 0.999
```

```
    },
    "masks": {
      "status": "byte_identical"
    },
    "index_sqlite": {
      "status": "numerically_equivalent",
      "hash_columns_deferred_to_payload_equivalence": 142,
      "vector_rows_compared": 3184
    }
  },
  "errors": [],
  "warnings": [],
  "infos": [
    {
      "code": "INFO_SQLITE_HASH_MISMATCH_ALLOWED_BY_SOURCE_LAYER_EQUIVALENCE",
      "severity": "info",
      "path": "/index/index.sqlite",
      "message": "SQLite hash columns differed exactly, but referenced payloads passed source-layer numerical equivalence."
    }
  ],
  "authenticity": []
}
```


Allowed strict-validation `status` values:

```
valid
valid_with_warnings
invalid
unreadable
```

Allowed equivalence-validation `status` values:

```
byte_identical
structurally_equivalent
numerically_equivalent
not_equivalent
unreadable
```

Allowed `authenticity_status` values:

```
not_checked
signature_missing
signature_verified
signature_mismatch
signature_untrusted
```

```
signature_unreadable
```

Allowed `severity` values:

```
info
warning
error
fatal
```

Allowed report buckets:

```
errors        Core validation failures and fatal Core unreadability findings
warnings      Core warnings that do not invalidate the package
infos         Informational Core or equivalence findings
authenticity  Detached-signature findings excluded from Core status computation
```

When `svp validate --equivalent` is run, the report MUST include a `metrics` object. If the result is `numerically_equivalent`, the metrics object MUST identify every tolerance-governed layer that differed, the metric used, the measured value, the threshold, and whether the layer passed or failed. A top-level `numerically_equivalent` result without per-layer metrics is not a conforming RC1 equivalence report.

RC1 defines this initial validation code registry. The authoritative machine-readable version MUST be published at `/spec/validation-codes.json`:

- `ERR_CORE_MISSING_MANIFEST` (`fatal`): `manifest.json` is absent or unreadable.
- `ERR_CORE_MISSING_SECTION` (`fatal`): a required core section is absent.
- `ERR_CORE_UNKNOWN_ROOT_SECTION` (`error`): a root-level section other than the allowed core sections or `/labels` exists.
- `ERR_CORE_MISSING_TRANSCRIPT` (`error`): transcript files are missing or invalid.
- `ERR_CORE_WORD_MISSING_SPEAKER` (`error`): a word record lacks exactly one primary `speaker_id` when `word_count` is nonzero.
- `ERR_CORE_TIMESTAMP_NONCANONICAL` (`error`): a stored timestamp does not match exact-rational round-half-to-even derivation.
- `ERR_CORE_RASTER_EXTENT_MISMATCH` (`error`): mask or depth extents do not match the package canonical analysis raster.
- `ERR_CORE_MISSING_DEPTH` (`error`): required depth index or block stream is missing.
- `ERR_CORE_MISSING_MASKS` (`error`): required mask index or block stream is missing.
- `ERR_CORE_INVALID_BLOCK_HEADER` (`error`): an SVP binary block header is malformed or unsupported.
- `ERR_CORE_INVALID_BLOCK_HASH` (`error`): a block payload, header, or block hash does not verify.
- `ERR_CORE_FORBIDDEN_BLOCK_TYPE` (`error`): a reserved, experimental, sentinel, or unknown block type appears in a strict v1.0 package.

- ERR_CORE_INDEX_SCHEMA_INVALID (error): index/index.sqlite does not contain the required schema.
- ERR_CORE_INDEX_MANIFEST_INVALID (error): index/index_manifest.json is missing, malformed, or fails schema validation.
- ERR_CORE_INDEX_LOGICAL_MISMATCH (error): SQLite single-package logical integrity failed, or exact-governed SQLite values failed during equivalence validation.
- ERR_CORE_HASH_MISMATCH (error): a required BLAKE3 digest does not match.
- ERR_CORE_PATH_TRAVERSAL (fatal): a ZIP entry path escapes the package root or is not normalized.
- ERR_SIGNATURE_MISMATCH (error, authenticity bucket): a provided .svpsig sidecar does not verify against the finalized .svp bytes.
- ERR_SIGNATURE_UNREADABLE (error, authenticity bucket): a provided .svpsig sidecar cannot be parsed or uses an unsupported required field.
- WARN_UNSUPPORTED_LABEL_SCHEMA (warning): /labels contains a future schema version the reader or validator does not support.
- WARN_CACHE_DISABLED (warning): the builder could not use the global cache and fell back to direct compute.
- WARN_JOURNAL_RETAINED_DIAGNOSTIC (warning): the journal was retained because the user explicitly requested diagnostic retention.
- WARN_SIGNATURE_UNTRUSTED (warning, authenticity bucket): a signature cryptographically verifies but the validator cannot establish trust in the provided key chain.
- WARN_SIGNATURE_NOT_PROVIDED (warning, authenticity bucket): no .svpsig was provided when authenticity verification was requested.
- INFO_DEGENERATE_VISUAL_SOURCE (info): empty entity tracks or near-uniform depth are valid for a documented degenerate visual source.
- INFO_SQLITE_HASH_MISMATCH_ALLOWED_BY_SOURCE_LAYER_EQUIVALENCE (info): a SQLite BLAKE3 hash column differed during equivalence comparison, but the referenced tolerance-governed payload passed its source-layer numerical equivalence rule.
- INFO_SIGNATURE_VERIFIED (info, authenticity bucket): a provided .svpsig verified against the exact finalized .svp byte stream.

Signature-related findings affect authenticity_status. They MUST NOT by themselves change a structurally valid SVP Core package from valid to invalid. A signed invalid package remains invalid, and an unsigned valid package remains valid.

19. Labels

Labels are the only non-core SVP v1.0 section.

labels/labels.jsonl MAY exist. If it exists, it maps IDs to interpretations.

```
{
  "id": "label_000001",
```

```
  "schema_version": "1.0-rc.1",
  "target_id": "entity_000 084",
  "label_type": "object_name",
  "value": "iPhone",
  "confidence": 0.91,
  "source": {
    "kind": "local_classifier",
    "processor_id": "proc_labeler_0001"
  }
}
```

Rules:

1. Labels MUST NOT be required for package validity.
2. Labels MUST NOT overwrite core entity records.
3. Labels MUST include provenance.
4. Human identity labels MUST NOT be written without explicit user action or an import source that records provenance.
5. schema_version is optional in RC1 label records. If absent, a reader MUST treat the label record as using the package svp_version label schema. If present and newer than the reader supports, Section 19.1 applies.

19.1 Forward compatibility for future label schemas

Because labels are non-core, unsupported future label schemas MUST NOT invalidate otherwise conforming SVP Core packages.

A reader that encounters /labels records with a schema_version newer than it supports MUST follow these rules:

1. It MUST NOT reject the package solely because /labels uses a newer schema version.
2. It MUST ignore unsupported label records or expose them as opaque non-core metadata.
3. It MUST NOT reinterpret unsupported label fields as core observations.
4. It MUST NOT allow unsupported labels to affect core package validation.
5. It SHOULD emit WARN_UNSUPPORTED_LABEL_SCHEMA indicating that labels were present but not fully interpreted.

A strict validator MUST report unsupported future labels as WARN_UNSUPPORTED_LABEL_SCHEMA. It MUST NOT report the package as core-invalid for that reason.

20. Reference processing pipeline

The reference CLI command is:

```
svp build input.mov --out input.svp
```

No network access is required. The reference builder MUST fail if it cannot produce every required core section.

20.1 Execution model

The reference builder is a deterministic asynchronous DAG compiler. It MUST NOT be implemented as one mandatory linear script.

The required build graph has these top-level lanes:

```
Input media fingerprint
  -> media inventory and package source preservation

Audio lane, CPU:
  -> extract original audio streams
  -> create analysis_mono_16k.wav
  -> generate waveform envelope
  -> run VAD
  -> run transcription with word timestamps
  -> run speaker diarization
  -> assign speaker IDs to words

Vision lane, decoder plus CPU plus accelerator:
  -> decode frame inventory with PTS
  -> detect shots
  -> estimate camera motion
  -> compute optical flow
  -> compute depth maps
  -> generate visual entity tracks
  -> generate masks
  -> generate spatial regions

Embedding lane:
  -> compute text embeddings after transcript chunks exist
  -> compute visual embeddings after shot keyframes and region crops exist
  -> compute speaker embeddings after diarization exists

Synchronization point:
  -> compute relationships after transcript timings, speaker segments,
     frames, shots, scenes, regions, masks, and depth exist

Package lane:
  -> build SQLite FTS and vector index
  -> write provenance
  -> validate package
```

```
-> atomically write final .svp
```

The relationship graph is the main synchronization point. It requires both language timing and spatial observation data. Everything before that point SHOULD run concurrently when dependencies allow.

20.2 Deterministic task model

Each DAG node is a task. Each task MUST have a stable deterministic ID.

Examples:

```
task.inventory.source_000
task.audio.extract.astream_000
task.vad.audio_chunk_000 014
task.transcript.audio_chunk_000 014
task.diarization.audio_chunk_000 014
task.frames.vstream_000.range_00 120 480_00 121 920
task.shots.vstream_000.pass_0001
task.depth.shot_000 421.frames_00 120 480_00 121 920
task.mask.shot_000 421.track_candidate_000 092
task.embedding.scene_text_000 044
task.relationships.scene_000 044
```

A task record contains:

```
{
  "task_id": "task.depth.shot_000 421.frames_00 120 480_00 121 920",
  "task_type": "depth",
  "depends_on": ["task.frames.vstream_000.range_00 120 480_00 121 920"],
  "processor_id": "proc_depth_0001",
  "model_refs": ["model_depth_anything_v2_small"],
  "input_refs": ["frame_00 120 480", "frame_00 121 920"],
  "output_refs": ["depth_00 120 480", "depth_00 121 920"],
  "cache_key": "b3:...",
  "parameters_blake3": "..."
}
```

Schedulers MAY choose any concurrency strategy, but output order is normative. JSONL records MUST be emitted in canonical order by section-specific sort keys, not by task completion order. Numeric floating-point outputs are compared under the Default Equivalence Profile defined in Section 5.16; cache reuse still requires exact BLAKE3 matches.

20.3 Content-addressable memoization

The reference builder MUST support a global content-addressable cache. Cache artifacts are not part of

SVP conformance. They are an implementation requirement for the official reference builder.

Default cache roots:

```
Windows: %LOCALAPPDATA%\SVP\Cache\v1
macOS:   ~/Library/Caches/org.svp/cache/v1
Linux:   ${XDG_CACHE_HOME}/svp/v1, or ~/.cache/svp/v1 when XDG_CACHE_HOME is unset
```

`SVP_CACHE_DIR` MAY override the root for development, CI, and render-farm operation.

A visual cache key MUST include at least the canonical SVP `model_id`, `model_bundle_id`, and `bundle_blake3`, not a registry slug or provider-specific name:

```
BLAKE3(
  "svp-cache-key-v1",
  svp_core_schema_version,
  processor_id,
  processor_version,
  model_id,
  model_bundle_id,
  bundle_blake3,
  model_blake3,
  model_runtime,
  execution_provider,
  normalized_input_format,
  time_range,
  frame_indices,
  decoded_pixel_blake3,
  processor_parameters_blake3
)
```

An audio cache key MUST include at least the canonical SVP `model_id`, `model_bundle_id`, and `bundle_blake3`, not a registry slug or provider-specific name:

```
BLAKE3(
  "svp-cache-key-v1",
  svp_core_schema_version,
  processor_id,
  processor_version,
  model_id,
  model_bundle_id,
  bundle_blake3,
  model_blake3,
  model_runtime,
  execution_provider,
  sample_rate,
  channel_layout,
```

```
  sample_format,
  audio_chunk_blake3,
  processor_parameters_blake3
)
```

A cache hit is valid only when:

1. The cache key matches.
2. The cached artifact BLAKE3 matches the artifact manifest.
3. The processor provenance matches the active processor contract.
4. The cached artifact validates against the active schema.

Cache hits MUST be recorded in the recovery journal and final provenance.

20.4 Recovery journal

A build that demands exhaustive analysis must survive interruption. The reference builder MUST create a recovery journal next to the requested output path.

For an output path `video.svp`, the journal path is:

```
video.svp-journal/
  build.sqlite
  journal_manifest.json
  blobs/
    pending/
    completed/
  locks/
    build.lock
```

The journal MUST be transactional. The reference implementation uses SQLite WAL mode for `build.sqlite` while the build is active.

Minimum required journal tables:

```
CREATE TABLE build_session (
  id TEXT PRIMARY KEY,
  started_utc TEXT NOT NULL,
  svp_version TEXT NOT NULL,
  builder_version TEXT NOT NULL,
  status TEXT NOT NULL
);

CREATE TABLE source_fingerprint (
  source_id TEXT PRIMARY KEY,
  path TEXT NOT NULL,
```

```
  size_bytes INTEGER NOT NULL,
  mtime_ns INTEGER,
  blake3 TEXT NOT NULL
);

CREATE TABLE task (
  task_id TEXT PRIMARY KEY,
  task_type TEXT NOT NULL,
  status TEXT NOT NULL,
  cache_key TEXT,
  started_utc TEXT,
  completed_utc TEXT,
  output_blake3 TEXT
);

CREATE TABLE task_dependency (
  task_id TEXT NOT NULL,
  depends_on_task_id TEXT NOT NULL,
  PRIMARY KEY (task_id, depends_on_task_id)
);

CREATE TABLE artifact (
  artifact_id TEXT PRIMARY KEY,
  task_id TEXT NOT NULL,
  relative_path TEXT NOT NULL,
  blake3 TEXT NOT NULL,
  byte_length INTEGER NOT NULL
);

CREATE TABLE artifact_provenance (
  artifact_id TEXT PRIMARY KEY,
  processor_id TEXT NOT NULL,
  parameters_blake3 TEXT NOT NULL,
  model_refs_json TEXT NOT NULL
);

CREATE TABLE cache_hit (
  task_id TEXT PRIMARY KEY,
  cache_key TEXT NOT NULL,
  artifact_id TEXT NOT NULL,
  verified INTEGER NOT NULL
);

CREATE TABLE failure (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  task_id TEXT,
  occurred_utc TEXT NOT NULL,
  reason TEXT NOT NULL
```

```
);
```

`svp build --resume` MUST read the journal, verify the source fingerprint, verify completed artifact hashes, reconstruct the DAG state, and continue from the last valid incomplete task. It MUST NOT blindly skip frames.

Completed task outputs MUST be written to a pending path, fsynced where supported, verified by BLAKE3, and then atomically moved into `blobs/completed/`.

20.5 Storage lifecycle

The reference builder MUST manage temporary and reusable storage so strict processing does not silently consume unbounded disk space.

20.5.1 Journal cleanup

After successful atomic package finalization and successful strict validation, the reference builder MUST delete the corresponding `.svp-journal` directory by default.

For an output path `video.svp`, `video.svp-journal/` MUST NOT remain after a successful default build. Retaining the journal after success without an explicit diagnostic request is a reference-builder conformance failure.

A user or CI system MAY request diagnostic journal retention. When diagnostic retention is requested:

1. The builder MAY keep the journal after success.
2. The validation report MUST include `WARN_JOURNAL_RETAINED_DIAGNOSTIC`.
3. The final `.svp` package MUST NOT contain the journal directory or any journal state.

20.5.2 Cache eviction

The reference builder MUST provide a global content-addressable cache with Least Recently Used eviction enabled by default.

Default cache policy:

```
maximum_size: 50 GiB
eviction: LRU
cache_key_hash: BLAKE3
artifact_hash: BLAKE3
```

If the cache exceeds the configured limit, the builder MUST automatically prune least-recently-used artifacts until the cache is under the limit. Eviction MUST respect active build locks and MUST NOT remove artifacts referenced by a live `.svp-journal`.

Cache read, write, permission, quota, corruption, or eviction failures MUST NOT fail the build. The builder MUST fall back to direct compute, continue package generation, and emit `WARN_CACHE_DISABLED` or a more specific non-fatal warning in the validation report and build logs.

Cache artifacts are not SVP package contents. A valid .svp MUST NOT depend on the continued existence of the global cache.

20.6 Visual entity tracker details

The v1 tracker is deterministic and label-free.

Steps:

1. Decode frames in presentation order.
2. For each shot, select a first keyframe and every Nth frame based on motion change.
3. Detect Shi-Tomasi corners and additional edge points.
4. Track points using pyramidal Lucas-Kanade optical flow.
5. Compute dense Farneback optical flow between adjacent frames.
6. Estimate global camera motion with RANSAC homography from stable background features.
7. Subtract global motion to get residual motion.
8. Cluster residual motion vectors into candidate moving regions.
9. Split or merge candidates using color histograms, depth discontinuities, and texture similarity.
10. Create candidate boxes and masks.
11. Propagate masks through frames using optical flow.
12. Smooth tracks using Kalman filtering.
13. Reacquire lost tracks using appearance histograms and predicted motion.
14. Write entity, track, region, mask, and relationship records.

The output is a visual observation graph, not a set of named objects.

20.7 Scene grouping

Scenes are grouped from shots using:

```
Temporal adjacency
Visual embedding similarity
Transcript embedding similarity
Audio continuity
Shot boundary strength
Speaker continuity
```

No LLM summarization is required.

20.8 Relationship computation

Relationships are produced by deterministic rules:

```
appears_in_shot: entity region overlaps shot interval
appears_in_scene: entity region overlaps scene interval
```

```
overlaps: mask IoU exceeds threshold
near: centroid distance below threshold
contains: one mask mostly contains another
occludes: masks overlap and source median inverse depth is greater
enters_frame: first visible region appears after absence
exits_frame: last visible region disappears before shot end
visible_during_speech: entity visible during speaker segment
moves_with: entities have correlated motion vectors
```

All thresholds are written to provenance.

20.9 OCR and visible-text processing

The reference builder MUST implement OCR as a distinct pipeline branch and MUST NOT hide OCR inside unrelated embedding, tracking, or label stages.

Required processor boundaries:

```
ocr_text_detection
ocr_text_recognition
ocr_layout_classification
ocr_numeric_extraction
ocr_text_region_color_sampling
ocr_index_integration
```

The OCR branch consumes frames, shots, scenes, regions, and entities as available. It emits text regions, text observations, numeric values, foreground/background color references when available, provenance, and index rows.

The builder MUST distinguish text detection, text recognition, number extraction, layout analysis, subtitles/captions, UI text, scene text/signage, chart/graph numbers, and timestamps/counters.

20.10 Structured color processing

The reference builder MUST implement color observations as a deterministic branch.

Required processor boundaries:

```
color_space_normalization
color_bucket_quantization
shot_color_summary
scene_color_summary
frame_color_summary
region_color_summary
entity_color_summary
text_region_foreground_background_color
```

```
color_index_integration
```

Color processing SHOULD run as soon as canonical raster frames, shot boundaries, scene boundaries, regions, entities, and text regions become available. Full-frame shot and scene summaries can run before OCR. Text-region foreground/background color extraction runs after OCR text regions exist.

21. Command-line surface

The first public artifact is a CLI. The CLI is the reference compiler, validator, inspector, query tool, and exporter.

Required commands:

```
svp build <input-media> --out <output.svp>
svp build <input-media> --out <output.svp> --resume
svp build <input-media> --out <output.svp> --fresh
svp validate <file.svp> --strict
svp validate <file.svp> --strict --report <report.json>
svp validate --equivalent <a.svp> <b.svp>
svp inspect <file.svp>
svp query <file.svp> <query-text>
svp export <file.svp> --format otio --out <timeline.otio>
svp dump <file.svp> --section <section-name>
svp serve <file.svp> --read-only --port 7867
svp models list
svp models install required-v1
svp models verify
svp models path
svp sign <file.svp> --out <file.svpsig>
svp verify-signature <file.svp> --signature <file.svpsig>
svp cache inspect
svp cache verify
svp cache prune
svp journal inspect <output.svp-journal>
```

`--resume` resumes from the recovery journal after verifying source fingerprints and completed artifact hashes.

`--fresh` ignores any existing recovery journal for the output path, but it MAY still use the global content-addressed cache if cache hits verify.

`svp validate --strict --report` MUST emit the RC1 validation report schema defined in Section 18.2.

`svp serve` exposes a local JSON-RPC API for apps and agents. It MUST default to `localhost` only.

`svp models install required-v1` installs the pinned Reference Model Set as `.svpmodel` bundles. `svp models verify` verifies bundle manifests, licenses, notices, and BLAKE3 hashes without running a build.

`svp sign` creates an optional detached `.svpsig` sidecar. `svp verify-signature` verifies an `.svpsig` against the exact finalized `.svp` byte stream and reports authenticity status without changing SVP Core validity.

Required JSON-RPC methods:

```
svp.search
svp.get_object
svp.get_time_range
svp.get_transcript_window
svp.get_regions
svp.get_masks
svp.get_depth_summary
svp.get_relationships
svp.export_otio
```

22. Agent usage model

An AI agent should not receive an entire video. It should query SVP.

Example flow:

```
Agent asks for moments where Speaker 1 says "camera"
and a foreground entity occupies more than 20 percent of frame.

SVP query engine returns time ranges, word IDs, entity IDs,
region IDs, confidence, and thumbnails if requested.

Agent proposes edit decisions using time ranges.

SVP export converts selected ranges to OTIO.

NLE opens OTIO or receives XML generated from OTIO.
```

The agent never needs to scrub the whole video frame by frame unless it asks for a targeted visual slice.

23. Validation rules

`svp validate --strict` MUST check:

1. The package is ZIP64 and contains an uncompressed `mimetype` first entry.
2. `manifest.json` exists and validates.
3. Every required section exists.
4. No unknown root-level section exists except `/labels`.
5. All JSON and JSONL files are UTF-8 and valid.
6. All IDs are unique.

7. All references resolve.
8. All time ranges are valid.
9. All core timestamps derived from source timebases match the exact rational round-half-to-even conversion defined in Section 7.
10. The canonical analysis raster is computed according to Section 12.1 and recorded in manifest.json.
11. Every frame record contains source dimensions, displayed dimensions, and analysis dimensions.
12. Every word has exactly one primary speaker_id unless word_count is zero.
13. Optional speaker_candidates entries, when present, resolve to valid speaker IDs and are ordered by descending confidence.
14. speech_overlap, when present and true, is consistent with VAD and diarization provenance.
15. transcript.json.language is an object with a valid primary, detected, mode, and confidence field set.
16. If VAD detects speech-like regions but word_count is zero, language.primary MUST be und and language.mode MUST be undetermined.
17. If media contains no linguistic content, language.primary MUST be zxx and language.mode MUST be none.
18. Word-level language metadata MUST NOT be required for SVP Core and MUST NOT be validated as a core field in v1.0.
19. Shot intervals cover the primary video stream without invalid overlap.
20. Scene intervals cover one or more shots.
21. Every entity has at least one track unless the package is a documented degenerate visual source.
22. Every non-empty track has regions.
23. Empty entities, entity_tracks, regions, masks.index, and visual relationship rows are valid only for degenerate visual sources as defined in Section 13.4.
24. Every region has a mask ref and depth ref.
25. Every mask block has a valid SVPB header, valid BLAKE3 hashes, valid dimensions, and decodes to the declared dimensions.
26. Mask block extents MUST match manifest.canonical_analysis_raster exactly.
27. A zero-length spatial/masks.blocks.svpmz stream is valid only when spatial/masks.index.jsonl contains zero records and no region references a mask.
28. Every depth block has a valid SVPB header, valid BLAKE3 hashes, valid dimensions, and decodes to the declared dimensions.
29. Depth block extents MUST match manifest.canonical_analysis_raster exactly.
30. Near-uniform depth is not a validation failure when the depth processor completed and provenance records support the output.
31. Every embedding record points into embeddings/embeddings.blocks.svpez, has the declared dimension, and resolves to a valid embedding block.
32. Binary block types MUST be valid strict v1.0 block types. Reserved, implementation-private, sentinel, and unknown block types MUST fail strict validation.

33. All .svpdz, .svpmz, and .svpez package entries use ZIP method STORE.
34. The SQLite index exists and contains required tables for words, shots, scenes, entities, regions, relationships, embeddings, and binary blocks. Degenerate visual packages MAY have zero rows in visual entity tables.
35. SQLite extension loading is disabled at read time.
36. index/index_manifest.json exists, validates against index-manifest.schema.json, and its logical_rows_blake3 matches the canonical logical row stream produced from index/index.sqlite by the Section 17.5 procedure. The SQLite index must also participate in the source-layer-aware comparison procedure defined in Section 17.5.
37. Provenance exists for every generated section.
38. provenance/validation.json exists in packages produced by the reference builder and follows Section 18.2.
39. BLAKE3 hashes match.
40. ZIP paths are normalized and cannot escape the package root.
41. The package contains no recovery journal, cache directory, temporary task outputs, scheduler state, or .svpsig sidecar inside the archive.
42. /labels, when present, is validated only if the label schema version is supported. Unsupported future label schemas produce WARN_UNSUPPORTED_LABEL_SCHEMA, not a core validation failure.
43. A detached .svpsig sidecar, when provided to the validator for authenticity verification, MUST verify against the exact finalized .svp package bytes or produce the appropriate authenticity validation code in the authenticity report bucket. Missing, unreadable, untrusted, or mismatched .svpsig files MUST NOT fail SVP Core validation.

svp validate --equivalent package_a.svp package_b.svp MUST use Section 5.16. It MUST report one of byte_identical, structurally_equivalent, numerically_equivalent, or not_equivalent.

When equivalence validation compares index/index.sqlite, it MUST use the source-layer-aware logical comparison procedure defined in Section 17.5. It MUST NOT compare raw SQLite file bytes for package equivalence.

When svp validate --equivalent is run, the validator MUST emit a metrics object in the validation report. If the result is numerically_equivalent, the metrics object MUST identify the layers that diverged and the measured deltas that passed within tolerance. A validator MUST NOT return numerically_equivalent without explaining which tolerance-governed layer triggered that status.

Equivalence validation MUST apply the same reporting rules everywhere in the package:

1. Exact-governed columns and payloads that differ produce not_equivalent.
2. Tolerance-governed payloads, vectors, and floating-point SQLite values that differ only within Section 5.16.2 thresholds produce numerically_equivalent, not not_equivalent.
3. SQLite hash columns that reference tolerance-governed payloads may mismatch only when the referenced payloads pass the source-layer equivalence rule.
4. SQLite hash columns that reference exact-governed bytes must match exactly.
5. A validator MUST include machine-readable details for any SQLite hash mismatch allowed by

source-layer equivalence.

Validation output MUST follow the report schema and code registry defined in Section 18.2. Core `status` and default validator exit code MUST be computed only from Core findings, not from `authenticity` findings. Validation output is written to `provenance/validation.json` for reference-builder package builds and to stdout or a caller-selected report path for external validation commands.

23.1 OCR validation rules

A strict validator MUST validate OCR and visible-text records with these checks:

1. `/text/` exists for video packages.
2. `text_regions.jsonl`, `text_observations.jsonl`, `numeric_values.jsonl`, and `text_absence.json` exist.
3. Every text region validates against `text-region.schema.json`.
4. Every text observation validates against `text-observation.schema.json`.
5. Every numeric value validates against `numeric-value.schema.json`.
6. Every OCR `observation_type` exists in `/spec/ocr-observation-types.json`.
7. Every bounding box is numeric, normalized bounds are within `[0, 1]`, pixel bounds are within the canonical analysis raster, and `x_min < x_max`, `y_min < y_max`.
8. Every text time range is valid and references existing frames, shots, scenes, regions, entities, or text regions when those references are present.
9. Confidence values are finite numbers in `[0, 1]`.
10. `normalized_text` MUST NOT be absent when `raw_text` is non-empty.
11. Numeric extraction MUST preserve the source text and MUST store numeric values as decimal strings.
12. OCR provenance IDs MUST reference valid processor provenance records.
13. The SQLite index MUST contain rows corresponding to text regions, text observations, and numeric values.

23.2 Color validation rules

A strict validator MUST validate color observations with these checks:

1. `/colors/` exists for video packages.
2. `color_observations.jsonl`, `color_summary.json`, and `color_absence.json` exist.
3. Every color observation validates against `color-observation.schema.json`.
4. `target_type` is registered and `target_id` references an existing scene, shot, frame, region, entity, or text region as declared.
5. `sampling_basis` is registered.
6. `color_space` exists in `/spec/color-spaces.json`.
7. `color_bucket_registry_version` exists in `/spec/color-buckets.json`.
8. Every bucket ID in `bucket_coverage` exists in the active color bucket registry.
9. Every bucket percentage is finite and within `[0, 1]`.
10. Bucket percentages MUST sum to `1.0` within the registry-defined tolerance.

11. `dominant_bucket` MUST be present in `bucket_coverage` and MUST be one of the maximum-coverage buckets after tolerance handling.
12. Color provenance IDs MUST reference valid processor provenance records.
13. The SQLite index MUST contain rows corresponding to color observations and bucket coverage values.

24. Security and privacy requirements

SVP packages are untrusted input.

Security requirements:

1. Readers MUST NOT execute package contents.
2. Readers MUST NOT follow external paths from inside the package without explicit user permission.
3. The builder MUST NOT auto-update models during a build.
4. The builder MUST NOT include telemetry in the reference implementation.
5. `svp serve` MUST bind to localhost by default.
6. The package MUST include hashes so tampering can be detected.
7. The validator MUST warn if labels contain human names, identity fields, or unsupported future label schemas.
8. Readers MUST defend against path traversal, oversized allocations, malformed SQLite files, and zip bombs.
9. Readers MUST treat packages as untrusted input.

24.1 Detached signature sidecars

RC1 defines `.svpsig` as an optional detached authenticity sidecar. A `.svpsig` file signs or otherwise authenticates the exact finalized bytes of a `.svp` package without modifying the ZIP archive.

The sidecar is intentionally outside SVP Core:

```
video.svp
video.svpsig
```

Rules:

1. A reader MUST be able to process a valid `.svp` package when no `.svpsig` exists.
2. A `.svpsig` MUST NOT appear inside the `.svp` ZIP archive.
3. A `.svpsig` MUST bind to the exact finalized `.svp` byte stream using `package_blake3`.
4. A `.svpsig` MUST NOT affect SVP Core structural validity.
5. If an `.svpsig` is present and authenticity verification is requested, a strict validator MAY verify it against a provided trust store or public key chain.
6. Signature verification MUST emit machine-readable authenticity validation codes defined in Section

18.2.

7. Signature findings MUST be reported in the `authenticity` report bucket and MUST NOT change Core `status` or the default validator exit code.
8. A valid signature over an invalid SVP package proves only that the invalid bytes were signed. It does not make the package conformant.
9. An invalid or untrusted signature over a valid SVP package affects authenticity reporting, not Core validity.

Minimum `.svpsig` JSON envelope:

```
{
  "schema_version": "svp-signature-1",
  "type": "raw-detached-signature",
  "package_filename": "video.svp",
  "package_blake3": "...",
  "signature_algorithm": "ed25 519",
  "signature_value": "base64:...",
  "signed_utc": "2026-06-19T00:00:00Z",
  "key_id": "...",
  "certificate_chain": []
}
```

Allowed `type` values in RC1:

```
raw-detached-signature
c2pa-manifest-sidecar
```

`raw-detached-signature` is the simple SVP-native mode. It signs the exact `.svp` bytes or a canonical signature payload containing `package_blake3` and package metadata.

`c2pa-manifest-sidecar` is reserved for C2PA-compatible manifest sidecar workflows. SVP does not redefine C2PA. It only defines how a reader or validator relates a sidecar to the exact `.svp` package bytes.

A strict validator SHOULD report the following authenticity codes as applicable:

```
INFO_SIGNATURE_VERIFIED
ERR_SIGNATURE_MISMATCH
ERR_SIGNATURE_UNREADABLE
WARN_SIGNATURE_UNTRUSTED
WARN_SIGNATURE_NOT_PROVIDED
```

25. Licensing decisions

Specification and schemas:

```
CC0-1.0
```

Reference implementation:

```
Apache-2.0
```

Reference model bundle policy:

```
Only models with permissive commercial-use-compatible licenses are allowed
in the official reference distribution.
Reference model bundles MUST include license files, notice files,
pinned revision identifiers, and BLAKE3 hashes for every model artifact.
```

Reference model choices:

```
Silero VAD: MIT
Depth Anything V2 Small: Apache-2.0
Nomic Embed Text v1.5: Apache-2.0
Nomic Embed Vision v1.5: Apache-2.0
SAM 2 is not required by the v1 reference processor
```

The reference distribution MUST pin exact model revisions. It MUST NOT rely on model-card license assumptions without packaging the exact license and notice files used for that release. The reference builder MUST NOT require Python to run the reference model set.

FFmpeg licensing must be handled carefully. The reference project should dynamically link to LGPL-compatible FFmpeg builds by default and document when GPL components are enabled.

26. Conformance classes

26.1 SVP Package

An SVP package conforms if it contains every required section, follows the exact package layout, passes JSON Schema validation, passes binary block validation, passes BLAKE3 integrity checks, and contains no forbidden root-level sections.

Unsupported future `/labels` records MUST NOT make an otherwise conforming SVP Core package invalid.

26.2 SVP Reader

An SVP reader conforms if it can:

1. Open valid packages.

2. Reject invalid packages.
3. Read `manifest.json`.
4. Resolve IDs.
5. Query transcript, time ranges, shots, scenes, entities, regions, depth summaries, masks, relationships, embeddings, and supported labels if present.
6. Ignore or expose unsupported future labels as opaque non-core metadata.
7. Parse SVP binary block headers.
8. Verify BLAKE3 payload and block integrity.
9. Disable SQLite extension loading.

26.3 SVP Builder

An SVP builder conforms if it creates packages that pass strict validation. The official reference builder additionally MUST implement deterministic DAG execution, global content-addressable memoization, and recovery journals.

26.4 SVP Validator

An SVP validator conforms if it implements all strict validation rules and exits non-zero for invalid packages. Default invalid-package exit behavior is based on SVP Core validity only. Authenticity findings from detached `.svpsig` sidecars MUST NOT cause a non-zero default exit when SVP Core is valid.

A validator MAY expose an explicit signature-enforcement mode that exits non-zero for signature mismatch, unreadable signature, or untrusted signature. Such a mode is not Core validation and MUST be documented as authenticity enforcement.

A validator that implements package equivalence comparison MUST use Section 5.16 and report `byte_identical`, `structurally_equivalent`, `numerically_equivalent`, or `not_equivalent`.

26.5 Release sequencing note

This note is non-normative.

SVP Reader, SVP Validator, SVP Inspector, and golden fixture packages may stabilize before the full deterministic-DAG SVP Builder is production-ready. This does not weaken SVP. It allows the standard to become testable before the heaviest reference compiler component ships.

Recommended release order:

1. svp-spec
2. svp-schemas
3. svp-fixtures
4. svp-validator
5. svp-reader
6. svp-inspector
7. svp-builder

The validator and fixture corpus are the early anchor. Without them, the standard is prose. With them, the standard becomes executable.

27. JSON Schema and registry policy

SVP v1.0 schemas MUST use JSON Schema 2020-12.

RC1 also requires normative machine-readable registry files under `/spec`:

```
/spec/validation-codes.json
/spec/equivalence-profile.json
/spec/block-types.json
/spec/color-buckets.json
/spec/color-spaces.json
/spec/ocr-observation-types.json
/spec/svp-model-bundle-v1.md
/spec/svp-signature-v1.md
```

Registry files MUST be versioned, reviewed, and tested with the same release process as JSON Schemas.

Every JSON and JSONL record type gets a schema file:

```
manifest.schema.json
waveform-record.schema.json
speech-region.schema.json
transcript.schema.json
word.schema.json
word-speaker-candidate.schema.json
speaker.schema.json
speaker-segment.schema.json
frame.schema.json
shot.schema.json
scene.schema.json
entity.schema.json
entity-track.schema.json
region.schema.json
mask-index-record.schema.json
depth-index-record.schema.json
relationship.schema.json
embedding-set.schema.json
embedding-index-record.schema.json
processor-record.schema.json
validation-report.schema.json
index-manifest.schema.json
model-bundle.schema.json
```

```
model-lock.schema.json
signature-sidecar.schema.json
label.schema.json
```

Schema rules:

1. `additionalProperties` MUST default to `false` for v1.0 core schemas.
2. Time fields MUST use integer microseconds.
3. ID fields MUST validate against prefix-specific regexes.
4. Controlled vocabularies MUST live in schema enums.
5. Schemas MUST include examples.
6. Conformance tests MUST include positive and negative examples for every schema.

28. Governance model

SVP should start as an open source standard repo:

```
github.com/svp-standard/svp
```

Repository structure:

```
/spec
  svp-v1.md
  validation-codes.json
  equivalence-profile.json
  block-types.json
/schemas
/reference
/conformance
/fixtures
/models
/samples
/docs
```

Governance rules:

1. The spec is versioned independently from the reference implementation.
2. No vendor gets special control over core sections.
3. Core required fields can only change in a major version.
4. Labels remain non-core unless a future major version proves a label type is observational rather than interpretive.
5. All spec changes require conformance test updates.
6. All generated records must keep provenance.

7. All sample packages must include license-clean media.
8. A steering group can merge spec changes only through public issues and pull requests.
9. Security reports use a private disclosure channel, then public advisories after fixes.

29. Risk register

29.1 File size

SVP files are larger than normal video files. This is expected. SVP trades storage for machine readability, searchability, and reuse. SVP mitigates size and rebuild pain with SVP binary block compression, ZIP `STORE` rules for precompressed payloads, content-addressable caching, and resumable journals.

29.2 Entity tracks are not semantic object labels

The reference tracker can produce `entity_000084` without knowing whether it is a phone, laptop, person, dog, or coffee mug. This is intentional.

29.3 Monocular depth is relative

Depth in v1.0 is relative inverse depth. It is not metric distance.

29.4 Speaker diarization can be wrong

Speaker segments are required but anonymous. Human names are labels.

29.5 Embedding models can age

Embedding model identity and hashes are required. Future major versions can update the reference embedding models.

29.6 Classical masks are imperfect

Masks are required observations, not magic truth. Confidence fields and provenance exist so readers can reason about quality.

29.7 Query results are evidence, not truth

SVP gives agents evidence. It does not make creative decisions by itself.

29.8 Cache poisoning and stale artifacts

Content-addressable memoization creates a new security and correctness surface. RC1 requires cache keys to include processor version, model identity, model hash, runtime, execution provider, normalized input format, time range, source chunk hash, and parameters. Cache hits MUST verify artifact hashes

before reuse.

29.9 DAG nondeterminism and floating-point variance

Parallel execution can produce nondeterministic completion order, and accelerator-backed floating-point processors can produce numerically tiny but byte-visible differences. RC1 requires canonical output ordering, deterministic task IDs, exact BLAKE3 reuse for cached artifacts, and a Default Equivalence Profile for comparing completed packages without pretending all derived arrays will be bit-identical.

29.10 Interrupted builds

Long builds can be interrupted. RC1 requires recovery journals so a failed build can resume from verified completed tasks rather than restarting the whole processing run.

29.11 Degenerate visual sources

Static cards, slates, and near-zero-motion footage can honestly produce no entity tracks. RC1 requires validators to distinguish between omitted processing and valid empty observations. Empty visual entity outputs are acceptable only when required files, depth outputs, provenance, and indexes still exist and validate.

29.12 Cache and journal disk growth

Strict processing can create large temporary and reusable artifacts. RC1 requires default journal cleanup after successful builds, a 50 GiB default LRU cache limit, active-build cache locks, and graceful fallback to direct compute when cache storage fails.

29.13 Detached signatures do not imply correctness

A valid signature proves that a signer signed the package bytes. It does not prove that the package is semantically correct, safe, complete, or Core-conformant. Validators MUST report Core validity and authenticity separately.

29.14 Model bundle supply chain

Reference Model Bundles reduce Python and dependency friction, but they create a supply-chain surface. RC1 requires signed, pinned, BLAKE3-verified bundles with included licenses and notices. Builders MUST NOT auto-update bundles during a build.

29.15 Timestamp drift

Long media files and fractional frame rates can expose timestamp drift if builders use floating-point math. RC1 forbids floating-point timestamp derivation and requires exact rational arithmetic with round-half-to-even conversion.

30. Why this direction works

SVP turns video into a queryable substrate. It does not ask an AI agent to watch a 40 GB file like a tiny intern trapped in a timeline goblin cave. It gives the agent a structured map:

```
What was said?
Who said it?
When was it said?
What shots exist?
What scenes exist?
What visual entities persist over time?
Where are they?
What masks define them?
What is closer or farther?
What relationships exist?
How can this be searched instantly?
```

That is the actual bottleneck in agentic video editing. The problem is not only editing. The problem is repeatedly rediscovering the same facts about the footage. SVP makes those facts portable.

31. Reference release target

The first reference release should be a CLI, not a GUI.

Required commands:

```
svp build input.mov --out input.svp
svp build input.mov --out input.svp --resume
svp validate input.svp --strict
svp validate input.svp --strict --report validation.json
svp validate --equivalent a.svp b.svp
svp inspect input.svp
svp query input.svp "speaker_0001 visible while speech is active"
svp export input.svp --format otio --out roughcut.otio
svp models list
svp models install required-v1
svp models verify
svp models path
svp sign input.svp --out input.svpsig
svp verify-signature input.svp --signature input.svpsig
svp cache verify
svp cache prune
svp journal inspect input.svp-journal
```

Minimum output:

```
Valid .svp package
All required core sections
Required transcript with word timestamps
Required speaker segments
Required shot and scene boundaries
Required entity track section, with zero records only for degenerate visual sources
Required mask section, with a zero-length mask stream only for degenerate visual sources
Required depth
Required embeddings
Required SQLite index
Required provenance
Required validation report
Required SVP binary block headers
```

Build requirements:

```
Deterministic asynchronous DAG execution
BLAKE3 content-addressable cache
Transactional recovery journal
Atomic final package write
Strict validation before success exit
Default Equivalence Profile support for package comparison
Canonical SQLite logical row-stream equivalence
Default journal cleanup after successful builds
LRU cache eviction with 50 GiB default limit
Machine-readable validation report and code registry
Per-layer equivalence metrics for --equivalent
Exact rational timestamp derivation
Optional detached .svpsig verification
Pinned SVP Model Bundle installation and verification
No Python runtime requirement for normal reference-builder operation
```

Desktop applications come later. The CLI is the standard's hammer, anvil, and tuning fork.

The Reader, Validator, Inspector, and fixture corpus SHOULD ship before or alongside the first full Builder preview so implementers can test packages even while the heaviest processing stack matures.

32. Release-candidate open issues

RC2 pauses Phase 03+ implementation until OCR and structured color observations are represented in schemas, registries, validation, index tables, fixtures, builder roadmap, and first-video-trial acceptance criteria. Remaining RC2 open issues should be treated as validator-blocking only if they prevent implementation of those artifacts.

RC1 resolved the final known validator-blocking ambiguities from Draft 0.6. RC2 adds OCR and structured

color observations as Core layers and pauses downstream implementation until those layers are represented in validator, fixtures, index, builder roadmap, and first-video acceptance checks.

At RC2, further changes should be restricted to:

```
contradiction fixes
validator-blocking ambiguity fixes
schema defects
fixture-driven conformance corrections
security footgun fixes
implementation impossibility fixes
editorial clarity and typo fixes
```

Remaining work before final v1.0:

1. Complete JSON Schema files for all record types and registry artifacts, including `index-manifest.schema.json`.
2. Build the golden fixture package corpus: static-card, silent-video, overlapping-speech, screen-recording, moving-object, empty-visual-tracks, mixed-language, undetermined-speech, no-linguistic-content, uniform-depth, vertical-video, square-video, ultrawide-video, non-square-pixel, fractional-timebase, signed-sidecar, signature-mismatch, model-bundle, and index-manifest fixtures.
3. Implement `svp validate --strict` against the fixture corpus.
4. Implement `svp validate --equivalent` against CPU, CoreML, DirectML, CUDA, and CPU-fallback fixture pairs where available.
5. Verify that direct payload equivalence and SQLite logical equivalence agree for `binary_blocks` hash columns and `vector_index` embedding values.
6. Finalize semantic versioning policy for index schema changes.
7. Finalize signature algorithms and trust-store requirements beyond the RC1 minimum sidecar envelope.
8. Decide whether the reference model registry uses static release assets, OCI artifacts, or both for `.svpmodel` distribution.
9. Finalize the exact `model-lock.json` schema and test disagreement handling against `model.svpmodel.json`.

No remaining open issue is allowed to weaken SVP Core for v1.0. If an issue requires weakening a required core section or field, it is deferred to a future major version.

33. References

[FFMPEG-DOCS] FFmpeg Documentation.
https://www.ffmpeg.org/documentation.html

[FFMPEG-FFPROBE] ffprobe Documentation.
https://ffmpeg.org/ffprobe.html

[FFMPEG-DOWNLOAD] FFmpeg Download Page, release notes snapshot accessed 2026-06-18.
https://www.ffmpeg.org/download.html

[OPENCV-OPTICAL-FLOW] OpenCV Optical Flow Tutorial.
https://docs.opencv.org/4.x/d4/dee/tutorial_optical_flow.html

[PYSCENEDETECT-DETECTORS] PySceneDetect Detectors API Documentation.
https://www.scenedetect.com/docs/latest/api/detectors.html

[WHISPERCPP] whisper.cpp GitHub Repository.
https://github.com/ggml-org/whisper.cpp

[SILERO-VAD] Silero VAD GitHub Repository.
https://github.com/snakers4/silero-vad

[SHERPA-ONNX-DIARIZATION] sherpa-onnx Speaker Diarization Documentation.
https://k2-fsa.github.io/sherpa/onnx/speaker-diarization/index.html

[DEPTH-ANYTHING-V2-SMALL] Depth Anything V2 Small Model Card.
https://huggingface.co/depth-anything/Depth-Anything-V2-Small

[NOMIC-TEXT] nomic-embed-text-v1.5 Model Card.
https://huggingface.co/nomic-ai/nomic-embed-text-v1.5

[NOMIC-VISION] nomic-embed-vision-v1.5 Model Card.
https://huggingface.co/nomic-ai/nomic-embed-vision-v1.5

[ONNX-RUNTIME-EP] ONNX Runtime Execution Providers.
https://onnxruntime.ai/docs/execution-providers/

[SQLITE-WAL] SQLite Write-Ahead Logging.
https://www.sqlite.org/wal.html

[SQLITE-FTS5] SQLite FTS5 Extension Documentation.
https://www.sqlite.org/fts5.html

[SQLITE-VEC] sqlite-vec GitHub Repository.
https://github.com/asg017/sqlite-vec

[BLAKE3] BLAKE3 Official Repository and Specification Pointers.
https://github.com/BLAKE3-team/BLAKE3

[PKWARE-APPNOTE] PKWARE ZIP Application Note.
https://support.pkware.com/pkzip/appnote

[OPENTIMELINEIO] OpenTimelineIO Documentation.
https://opentimelineio.readthedocs.io/en/latest/

[JSON-SCHEMA] JSON Schema Specification.
https://json-schema.org/specification

[ZSTD-RFC8878] RFC 8878: Zstandard Compression and the application/zstd Media Type.
https://datatracker.ietf.org/doc/html/rfc8878

[ISO639-2-LOC] Library of Congress ISO 639-2 Code List.
https://www.loc.gov/standards/iso639-2/php/code_list.php

[C2PA-SPEC] C2PA Technical Specification.
https://spec.c2pa.org/specifications/specifications/2.2/specs/C2PA_Specification.html

[ONNX-RUNTIME-INSTALL] ONNX Runtime Install Documentation.
https://onnxruntime.ai/docs/install/

[RFC2119] RFC 2119: Key words for use in RFCs to Indicate Requirement Levels.
https://datatracker.ietf.org/doc/html/rfc2119

[RFC8174] RFC 8174: Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words.
https://datatracker.ietf.org/doc/html/rfc8174